Guía práctica: API RESTful y Front-end de CuscoSmart IA

Guía turística inteligente del Cusco

Node.js + Express + MySQL (procedimientos almacenados) + JWT · Cliente: React + Vite + Leaflet

Datos del proyecto	
Tipo	Proyecto **grupal**
Integrantes	Gabriel Paul Alata Condori Rodrigo Ccahuana Mayo Elizabeth Flores Vargas
Curso	Fullstack Developer Software
Institución	SENATI
Fecha	07 de noviembre de 2026
Repositorios	github.com/paulgabriel1/cuscosmart-api github.com/paulgabriel1/cuscosmart-web

Cada función incluye su **evidencia**: captura real de la petición a la API o de la pantalla de la web.

Contenido

1. Qué vas a construir y con qué
2. Autenticación: por qué JWT
3. Base de datos y diagrama ER
4. Reglas de negocio
5. Diseño de endpoints y permisos
6. Preparar el entorno en Windows
7. Estructura del proyecto
8. Errores, validación y respuestas
9. Catálogo de atractivos
10. Favoritos y viajes ordenados por IA
11. Reseñas verificadas y recorrido
12. Asistente de preguntas frecuentes
13. Administración y fotos
14. Seguridad
15. Pruebas con Postman y Swagger
16. Front-end: qué se construye y con qué
17. Conectar React con la API
18. Pantallas y funciones del turista
19. Panel del administrador
20. Modo oscuro, idioma y celular
21. Cómo ejecutarlo paso a paso
22. Resumen de pruebas y checklist
23. Historial de Git por fases
24. Errores frecuentes

PARTE I · API REST

1. Qué vas a construir y con qué

Una API REST bajo la ruta `/api/v1` que gestiona **atractivos turísticos** del Cusco (con categorías, zonas, horarios y fotos), **favoritos**, **viajes** cuyas paradas ordena la IA de la más cercana a la más lejana, **reseñas verificadas** (solo de lugares visitados) y un **asistente** de preguntas frecuentes. Tiene dos roles: `TURISTA` y `ADMIN`. Se prueba con Postman y la consume la web en React.

Pieza	Elección	Por qué
Runtime	Node.js LTS (20 o superior)	Estable y con `--watch` integrado.
Framework	Express 5	Simple y captura solo los errores de funciones `async`.
Base de datos	MySQL / MariaDB (XAMPP)	11 tablas, **61 procedimientos almacenados**, 4 triggers y 1 función. Las reglas viven en la base.
Acceso a MySQL	mysql2, solo `CALL`	La API nunca arma SQL: ejecuta procedimientos con parámetros (sin inyección SQL).
Autenticación	JWT + bcryptjs (+ Google opcional)	Ver sección 2.
Validación	zod	Esquemas legibles y mensajes claros por campo.
Seguridad	helmet, cors, express-rate-limit	Cabeceras seguras, CORS para la web y freno a la fuerza bruta.
Imágenes	multer	Subir fotos desde la PC; se verifica el contenido real del archivo.
Documentación	Swagger (OpenAPI 3)	Probar cada endpoint desde el navegador.
Pruebas	Postman con scripts de test	149 peticiones y 456 pruebas automáticas.

Arquitectura en capas

```
Petición → routes → middlewares (token, rol, validación) → controller → service → utils/sp.js → MySQL (CALL)
```

- **routes:** la URL y quién puede entrar.
- **controller:** lee la petición, llama al service y arma la respuesta HTTP.
- **service:** llama a los procedimientos almacenados; es el único que habla con la base.

Regla de oro: el cliente nunca envía totales, distancias, órdenes ni el id del usuario. La API los calcula o los toma del token.

2. Autenticación: por qué JWT

- **Sesiones:** guardan estado en el servidor y usan cookies; es incómodo con una web React separada.
- **JWT (elegido):** al iniciar sesión el servidor firma un token; la web lo envía en cada petición con `Authorization: Bearer <token>`. Dura 8 horas.
- **Google (opcional):** la API verifica la firma del token de Google con su librería oficial y crea la cuenta si no existe.

Las contraseñas se guardan con **bcrypt**. El login responde lo mismo si falla el correo o la contraseña, y solo cuenta los intentos **fallidos** por correo (10 cada 15 minutos).

> **Evidencia:** login correcto devuelve el token y los datos del usuario (sin la contraseña).

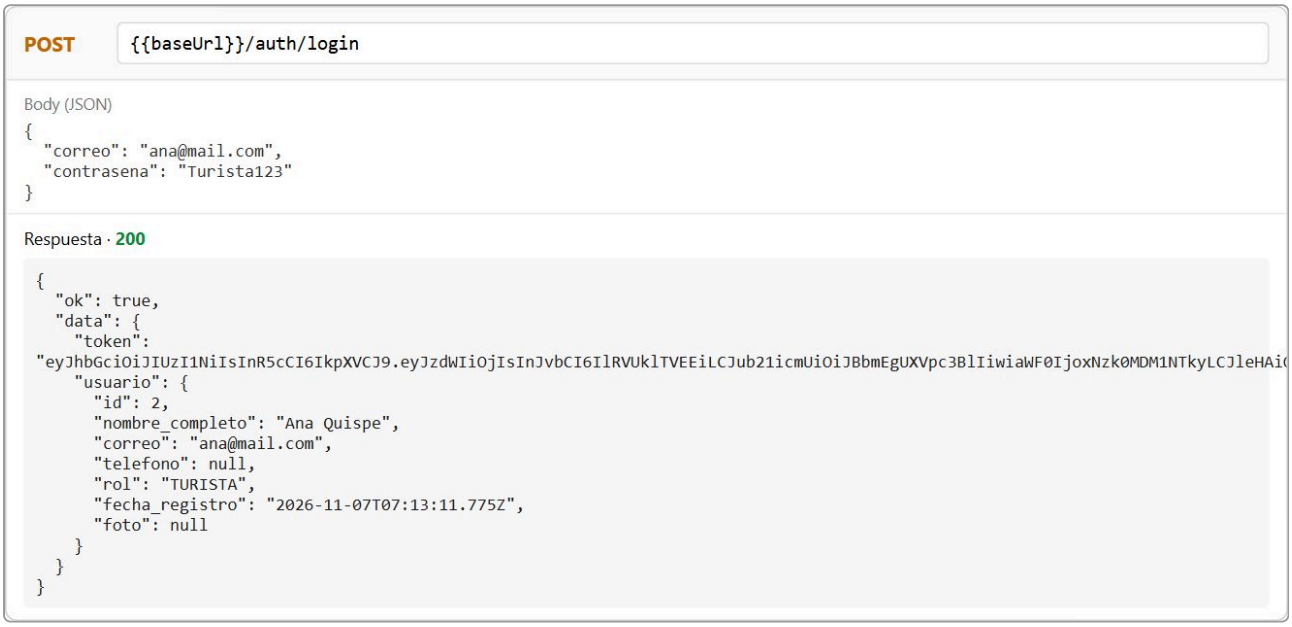


Figura 1. `POST /auth/login` → 200 con token JWT.



Figura 2. `GET /auth/me` con el token: perfil y totales.

3. Base de datos y diagrama ER

La base `cuscosmart_db` tiene 11 tablas. Toda la lógica de datos (validaciones, cálculos y reglas) está en **procedimientos almacenados** y **triggers**; la API solo los llama.

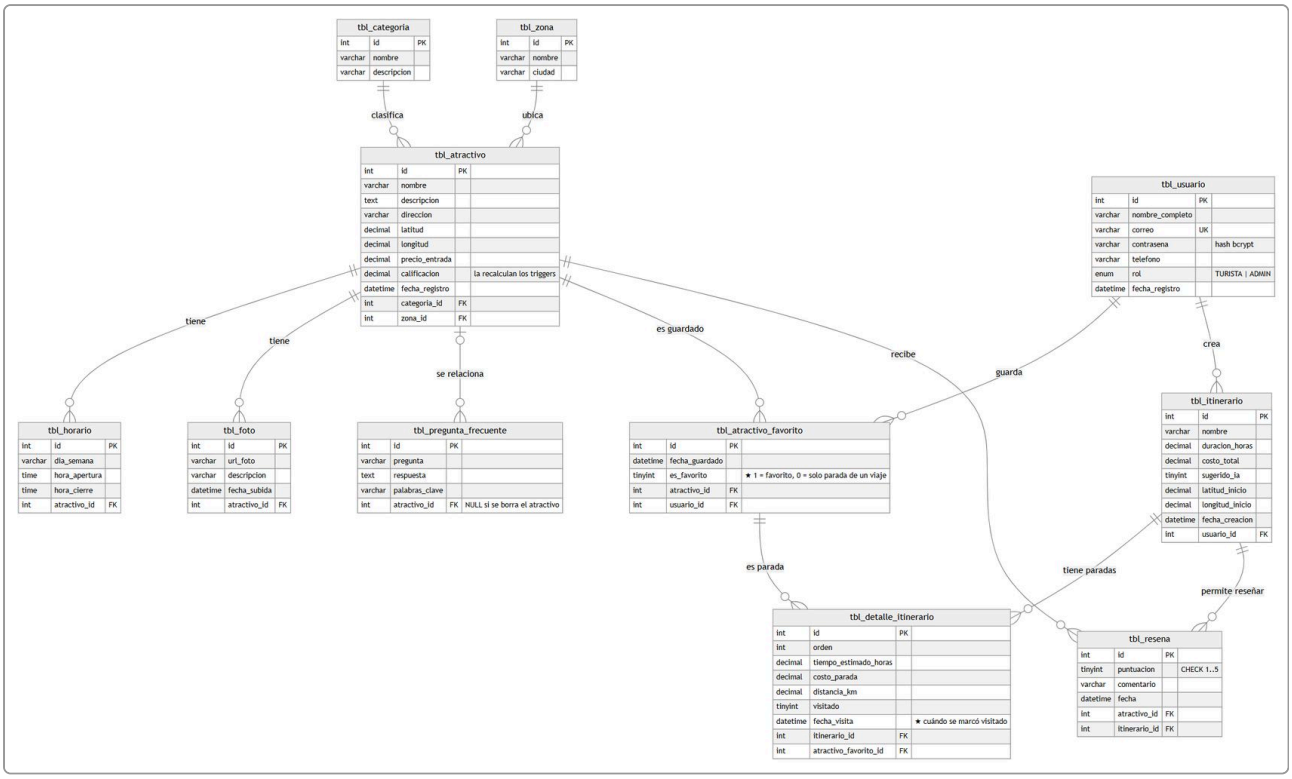


Figura 3. Diagrama entidad-relación (11 tablas y 12 llaves foráneas).

Archivo	Para qué sirve
`schema.sql`	Crea la base y las 11 tablas con llaves, UNIQUE y CHECK.
`logica.sql`	Función Haversine, 4 triggers y 61 procedimientos.
`seed.sql`	Datos de prueba: 27 atractivos, 6 categorías, 3 zonas, 20 preguntas y 2 usuarios.
`usuario.sql`	Usuario de MySQL de la API, con permiso solo de `EXECUTE`.
`migracion.sql`	Actualiza bases antiguas sin borrar datos.

> **Evidencia:** `npm run db:reset` crea y verifica la base.

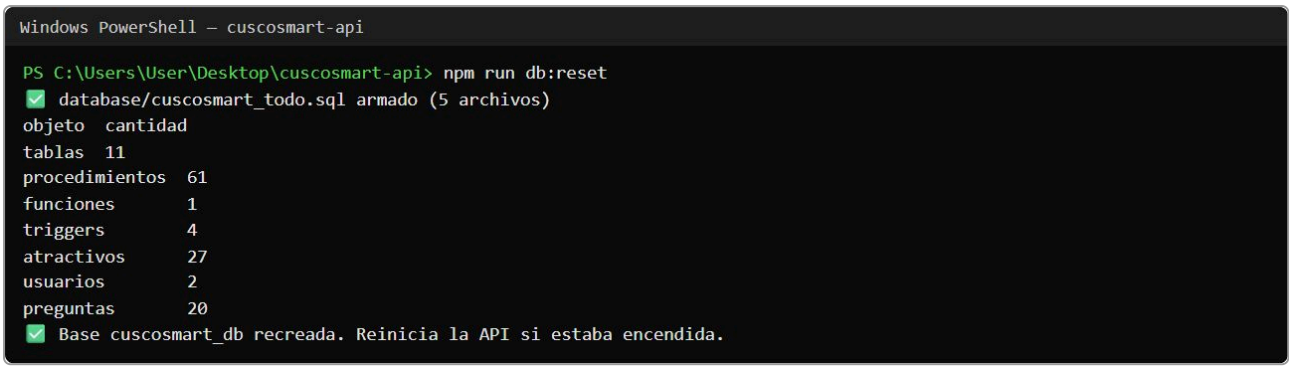


Figura 4. Conteo de objetos creados.

4. Reglas de negocio

ID	Regla	Dónde se aplica
RN-01	El correo es único.	`sp_registrar_usuario` + UNIQUE
RN-02	Contraseña cifrada; mínimo 8 caracteres, con letras y números.	service + zod
RN-03	Solo el ADMIN gestiona catálogo, preguntas y usuarios, y modera reseñas.	`authorize('ADMIN')`
RN-04	No se borra una categoría o zona con atractivos.	llave foránea → 409
RN-05	No se repiten nombres de categorías ni zonas.	procedimientos
RN-06	La hora de cierre es posterior a la de apertura.	`sp_agregar_horario`
RN-07	Un atractivo está una vez en los favoritos de cada turista.	procedimiento + UNIQUE
RN-08	Cada turista solo ve y cambia lo suyo; lo ajeno responde 404.	usuario del token
RN-09	Un lugar aparece una sola vez por viaje.	`sp_agregar_parada` + UNIQUE
RN-10	Duración, costo y orden de paradas los calcula el sistema.	`sp_actualizar_totales`
RN-11	Ordenar con IA exige punto de inicio; ordena por distancia (Haversine).	`sp_ordenar_itinerario_ia`
RN-12	Solo se reseña un lugar **visitado**, con puntuación de 1 a 5.	trigger `trg_resena_valida` + CHECK

RN-13	La calificación es el promedio de reseñas y se recalcula sola.	triggers de calificación
RN-14	El ADMIN no puede quitarse su rol ni borrar su cuenta.	service + `authenticate`
RN-15	Límites: 300 peticiones/min; 10 logins fallidos por correo cada 15 min.	`limitadores.js`
RN-16	Las fotos subidas deben ser imágenes reales de hasta 5 MB.	`utils/imagenes.js`

5. Diseño de endpoints y permisos

Base: `http://localhost:3000/api/v1`. Público = sin token · Sesión = con token · ADMIN = solo administrador.

Recurso	Endpoints principales	Acceso
Sesión	`POST /auth/registro`, `POST /auth/login`, `POST /auth/google`, `GET·PUT /auth/me`, `PUT·DELETE /auth/me/foto`	Público / Sesión
Catálogo	`GET /categorias`, `/zonas`, `/atractivos`, `/atractivos/cercanos`, `/atractivos/:id`	Público (escribir: ADMIN)
Fotos y horarios	`POST /atractivos/:id/fotos/subir`, `POST …/fotos/url`, horarios	ADMIN
Favoritos	`GET·POST /favoritos`, `DELETE /favoritos/:id`	Sesión
Viajes	`/itinerarios`: crear, paradas, `ordenar-ia`, visitado, reseñas, `recorrido`	Sesión
Reseñas	`GET /atractivos/:id/resenas`, borrar la propia, moderar	Público / Sesión / ADMIN
Asistente	`POST /asistente/preguntar`, `/preguntas-frecuentes`	Público (gestionar: ADMIN)
Usuarios	`GET /usuarios`, `PATCH /usuarios/:id/rol`, `DELETE /usuarios/:id`	ADMIN

En total son **61 rutas**, todas documentadas en Swagger.

6. Preparar el entorno en Windows

Herramienta	Dónde	Cómo comprobar
Node.js LTS	nodejs.org	`node -v` y `npm -v`
XAMPP (MySQL)	apachefriends.org	MySQL en verde en el panel

Git	git-scm.com	`git --version`
VS Code y Postman	code.visualstudio.com · postman.com	Abren sin error

7. Estructura del proyecto

Igual que el proyecto de referencia del curso: **un archivo por recurso** en cada capa.

```
cuscosmart-api/
├── database/        schema.sql · logica.sql · seed.sql · usuario.sql · migracion.sql · cuscosmart_todo.sql
├── docs/            INSTALACION.md · diagrama-er.md · guia/ (fases 0 a 13)
├── postman/         cuscosmart.collection.json · local.environment.json
├── scripts/         armarSql · migrar · generarPostman · probarColeccion · verificarDocs · crearAdmin
└── src/
    ├── app.js · server.js
    ├── config/      env.js (valida el .env) · db.js (pool de conexiones)
    ├── middlewares/ auth.js · validate.js · errorHandler.js · limitadores.js · subidas.js
    ├── routes/      index.js + un archivo por recurso
    ├── controllers/ un archivo por recurso
    ├── services/    un archivo por recurso
    ├── schemas/     un archivo por recurso + comunes.js
    ├── utils/       sp.js (única puerta a MySQL) · imagenes.js · AppError.js
    └── docs/        openapi.js (Swagger)

Recursos: auth · categorias · zonas · atractivos · favoritos · itinerarios
          resenas · preguntas · asistente · usuarios
Ejemplo:  routes/atractivos.routes.js → controllers/atractivos.controller.js
          → services/atractivos.service.js → CALL sp_listar_atractivos(...)
```

8. Errores, validación y respuestas

Todas las respuestas tienen el mismo formato. Los procedimientos avisan los errores con `SIGNAL SQLSTATE` y el manejador de errores los traduce a códigos HTTP.

```
{ "ok": true,  "data": { … }, "meta": { "page": 1, "limit": 10, "total": 27, "totalPages": 3 } }
{ "ok": false, "error": { "code": "BUSINESS_RULE", "message": "…" } }
```

HTTP	code	Cuándo
400	VALIDATION_ERROR	Datos mal enviados (zod)
401	NO_TOKEN · INVALID_TOKEN	Sin sesión o token inválido
403	FORBIDDEN	El rol no tiene permiso
404	NOT_FOUND	No existe o no es tuyo
409	DUPLICATE · IN_USE	Repetido o en uso
422	BUSINESS_RULE	Rompe una regla de negocio

429	TOO_MANY_REQUESTS	Demasiados intentos

Evidencia: cada error controlado con su código.

POST {{baseUrl}}/auth/registro

Body (JSON)

```
{
  "nombre_completo": "Lu",
  "correo": "no-es-correo",
  "contrasena": "123"
}
```

Respuesta · 400

```
{
  "ok": false,
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "Datos inválidos",
    "details": [
      {
        "campo": "nombre_completo",
        "mensaje": "Mínimo 3 caracteres"
      },
      {
        "campo": "correo",
        "mensaje": "Correo inválido"
      },
      {
        "campo": "contrasena",
        "mensaje": "Mínimo 8 caracteres"
      },
      {
        "campo": "contrasena",
        "mensaje": "Debe tener al menos una letra"
      }
    ]
  }
}
```

Figura 5. 400: el detalle dice qué campo falla.

GET {{baseUrl}}/favoritos

Respuesta · 401

```
{
  "ok": false,
  "error": {
    "code": "NO_TOKEN",
    "message": "Token requerido"
  }
}
```

Figura 6. 401: sin token.

POST {{baseUrl}}/categorias Bearer token

Body (JSON)

```
{
  "nombre": "Prueba"
}
```

Respuesta · 403

```
{
  "ok": false,
  "error": {
    "code": "FORBIDDEN",
    "message": "No tienes permiso para esta acción"
  }
}
```

Figura 7. 403: una turista no puede crear categorías.



Figura 8. 404: el viaje no existe o es de otra persona.



Figura 9. 409: categoría repetida.

9. Catálogo de atractivos

Lectura pública: lista paginada con filtros, búsqueda tolerante a errores de tipeo, lugares cercanos por distancia y detalle con horarios y fotos.



Figura 10. `GET /atractivos?page=1&limit=3` : datos y `meta` de paginación.

GET `{{baseUrl}}/atractivos?buscar=sacsaywaman&limit=3`

Respuesta · **200**


```
{
  "ok": true,
  "data": [],
  "meta": {
    "page": 1,
    "limit": 3,
    "total": 0,
    "totalPages": 0,
    "sugerencias": [
      {
        "id": 1,
        "nombre": "Sacsayhuamán",
        "categoria": "Sitio arqueológico"
      }
    ]
  }
}
```


Figura 11. Búsqueda con error de tipeo ("sacsaywaman"): la API sugiere Sacsayhuamán.

GET `{{baseUrl}}/atractivos/cercanos?lat=-13.5167&lng=-71.9788&radio_km=0.6&limit=5`

Respuesta · **200**


```
{
  "ok": true,
  "data": [
    {
      "id": 5,
      "nombre": "Plaza de Armas",
      "latitud": -13.5167,
      "longitud": -71.9788,
      "precio_entrada": 0,
      "calificacion": 0,
      "categoria": "Barrio y mercado",
      "zona": "Centro Histórico",
      "distancia_km": 0
    },
    {
      "id": 3,
      "nombre": "Catedral del Cusco",
      "latitud": -13.5163,
      "longitud": -71.9785,
      "precio_entrada": 25,
      "calificacion": 5,
      "categoria": "Iglesia",
      "zona": "Centro Histórico",
      "distancia_km": 0.06
    },
    {
      "id": 16,
      "nombre": "Iglesia de la Compañía de Jesús",
      "latitud": -13.5173,
      "longitud": -71.978,
      "precio_entrada": 15,
      "calificacion": 0,
  … (28 líneas más)
```


Figura 12. `GET /atractivos/cercanos` : ordenados por distancia (Haversine).

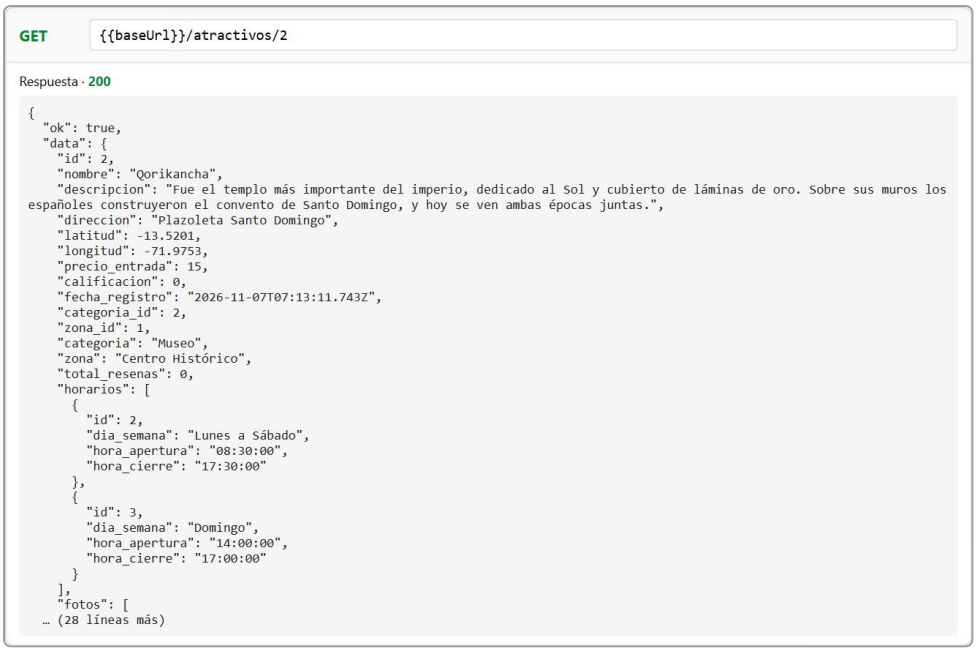


Figura 13. Detalle con horarios y fotos.

10. Favoritos y viajes ordenados por IA

El turista guarda lugares, crea un viaje con su punto de partida (por ejemplo, el hotel) y agrega paradas. La "IA" ordena las paradas de la más cercana a la más lejana con la función `fn_distancia_km` (Haversine) y recalcula duración y costo.



Figura 14. `POST /favoritos`.

POST {{baseUrl}}/itinerarios Bearer token

Body (JSON)

```
{
  "nombre": "Tarde en el centro",
  "latitud_inicio": -13.5175,
  "longitud_inicio": -71.98
}
```

Respuesta · 201


```
{
  "ok": true,
  "data": {
    "id": 2,
    "nombre": "Tarde en el centro",
    "turista": "Ana Quispe",
    "duracion_horas": 0,
    "costo_total": 0,
    "sugerido_ia": false,
    "latitud_inicio": -13.5175,
    "longitud_inicio": -71.98,
    "fecha_creacion": "2026-11-07T07:13:12.813Z",
    "paradas": []
  }
}
```


Figura 15. `POST /itinerarios` *: nuevo viaje con punto de inicio.*



Figura 16. `POST /itinerarios/:id/ordenar-ia` *: paradas ordenadas, con distancia, duración y costo.*

11. Reseñas verificadas y recorrido

Para evitar reseñas falsas, solo se reseña un lugar marcado como **visitado** (lo exige el trigger `trg_resena_valida`). La calificación del atractivo se recalcula sola.

POST `{{baseUrl}}/itinerarios/2/resenas` Bearer token

Body (JSON)

```
{
  "atractivo_id": 16,
  "puntuacion": 5
}
```

Respuesta · **422**

```
{
  "ok": false,
  "error": {
    "code": "BUSINESS_RULE",
    "message": "Solo puedes reseñar atractivos que visitaste en tu itinerario"
  }
}
```

Figura 17. 422: intentar reseñar sin haber visitado.

PATCH `{{baseUrl}}/itinerarios/2/paradas/6/visitado` Bearer token

Respuesta · **200**


```
{
  "ok": true,
  "data": {
    "id": 2,
    "nombre": "Tarde en el centro",
    "turista": "Ana Quispe",
    "duracion_horas": 3,
    "costo_total": 85,
    "sugerido_ia": true,
    "latitud_inicio": -13.5175,
    "longitud_inicio": -71.98,
    "fecha_creacion": "2026-11-07T07:13:12.813Z",
    "paradas": [
      {
        "parada_id": 6,
        "orden": 1,
        "distancia_km": 0.22,
        "tiempo_estimado_horas": 1,
        "costo_parada": 15,
        "visitado": true,
        "fecha_visita": "2026-11-07T07:13:12.872Z",
        "favorito_id": 6,
        "es_favorito": false,
        "atractivo_id": 16,
        "atractivo": "Iglesia de la Compañía de Jesús",
        "latitud": -13.5173,
        "longitud": -71.978,
        "categoria": "Iglesia",
        "foto": "img/compania-jesus.jpg",
        "resena_id": null,
        "puntuacion": null,
        "resenado": false
  … (44 líneas más)
```


Figura 18. Marcar la parada como visitada.

POST `{{baseUrl}}/itinerarios/2/resenas` Bearer token

Body (JSON)

```
{
  "atractivo_id": 16,
  "puntuacion": 5,
  "comentario": "Hermosa por dentro, vale la pena la visita."
}
```

Respuesta · **201**

```
{
  "ok": true,
  "data": {
    "id": 2,
    "puntuacion": 5,
    "comentario": "Hermosa por dentro, vale la pena la visita.",
    "fecha": "2026-11-07T07:13:12.878Z",
    "atractivo_id": 16,
    "atractivo": "Iglesia de la Compañía de Jesús",
    "nueva_calificacion": 5
  }
}
```

Figura 19. Reseña aceptada y nueva calificación.



Figura 20. `GET /itinerarios/recorrido` *: visitados y por visitar.*

12. Asistente de preguntas frecuentes

Busca la pregunta con un índice **FULLTEXT** de MySQL. Si no encuentra respuesta, sugiere preguntas parecidas. No guarda lo que pregunta el usuario.

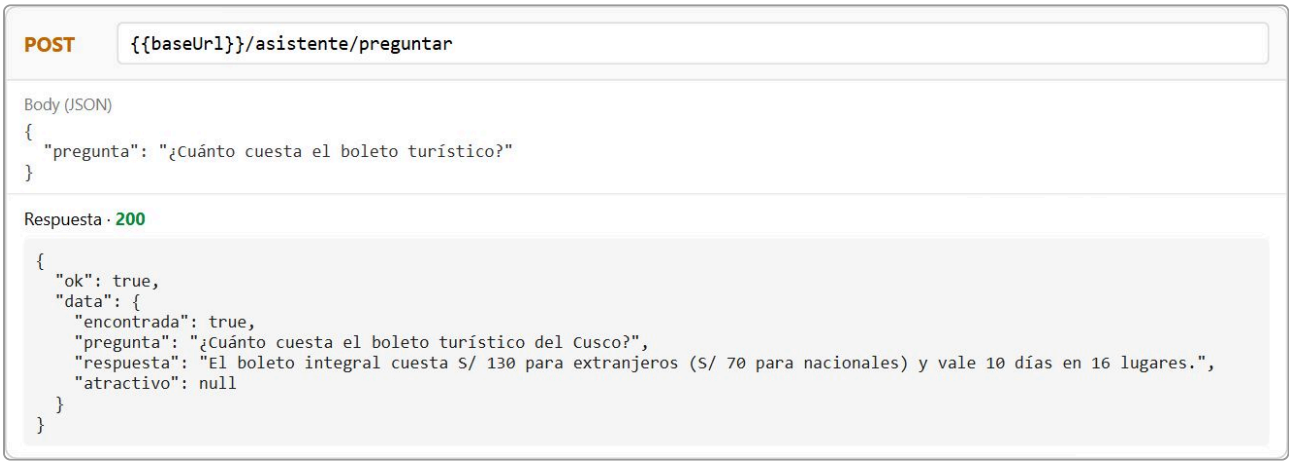


Figura 21. `POST /asistente/preguntar`.

13. Administración y fotos

El ADMIN gestiona el catálogo, las preguntas y los usuarios. Las fotos se suben desde la PC (hasta 5 a la vez) o desde un link: la API descarga la imagen, revisa que sea una imagen real y, si el link es una página, usa su foto principal.



Figura 22. `GET /usuarios` *(solo ADMIN).*



Figura 23. `POST /categorias` *: 201 creada.*

14. Seguridad

- La API entra a MySQL con un usuario que **solo ejecuta procedimientos**: no lee ni borra tablas directamente.
- Sin SQL armado con texto: todo es `CALL` con parámetros.
- Token con algoritmo fijo (HS256); el rol se lee de la base en cada petición.
- Lo ajeno responde 404, para no revelar que existe.
- helmet, CORS limitado a la web, límite de tamaño del JSON y `.env` fuera de Git.
- Fotos: se revisa la firma del archivo y no se descargan links de la red interna (SSRF).

15. Pruebas con Postman y Swagger

La colección `postman/cuscosmart.collection.json` tiene 149 peticiones en 7 carpetas, cada una con sus pruebas automáticas (código de estado, formato y datos). Se corre con `npm run test:api` o con **Run collection** en Postman.

> **Evidencia: 149 peticiones · 456 pruebas aprobadas · 0 fallidas.**

```
Windows PowerShell — cuscosmart-api

PS C:\Users\User\Desktop\cuscosmart-api> npm run test:api
== 00 Autenticación
== 01 Catálogo
== 02 Favoritos
== 03 Itinerarios e IA
== 04 Asistente y preguntas frecuentes
== 05 Administración
== 06 Seguridad
Peticiones: 149 · Pruebas: 456 aprobadas, 0 fallidas
```

Figura 24. Resultado de `npm run test:api`.

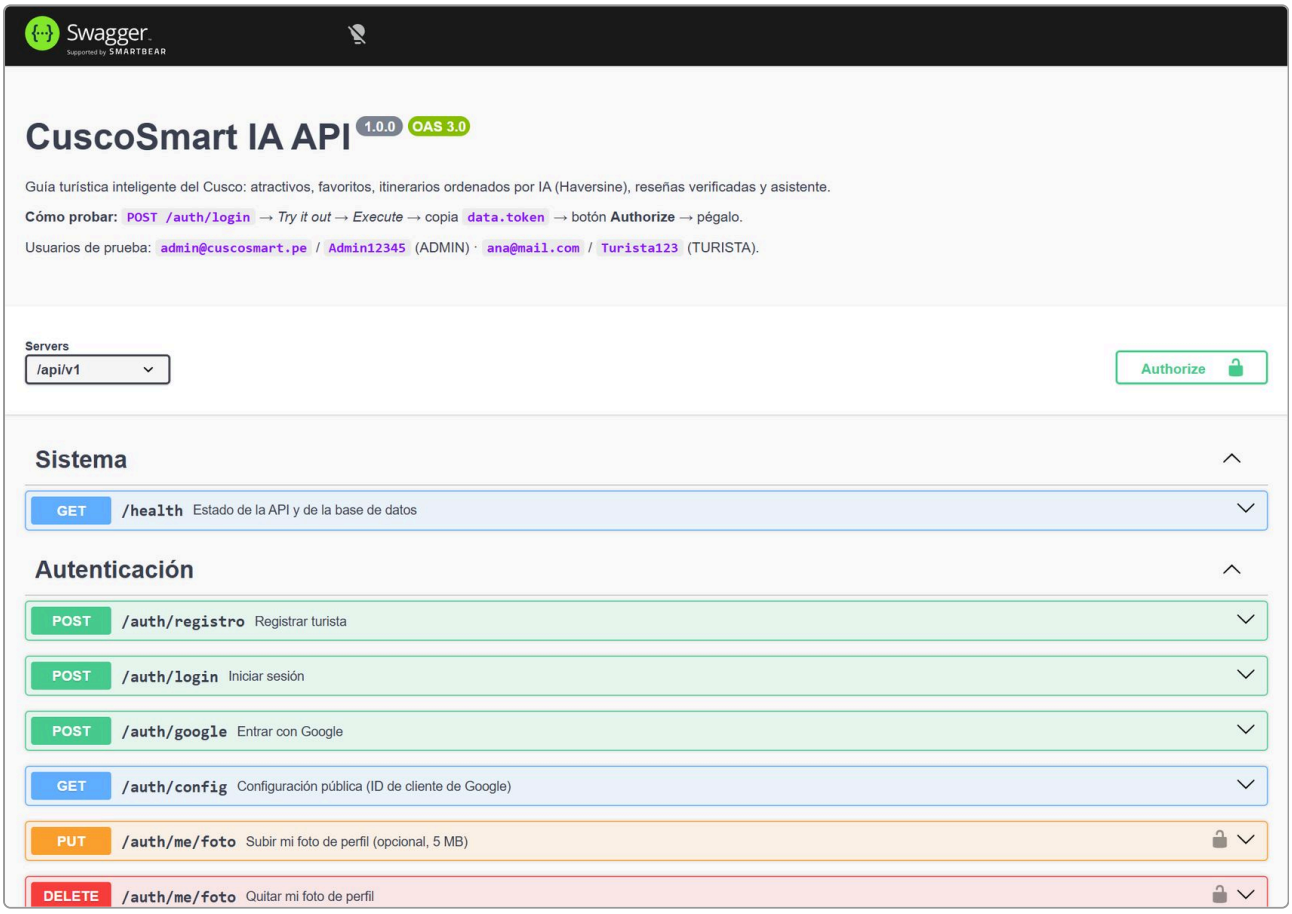


Figura 25. Swagger en `/api/v1/docs`: las 61 rutas documentadas.

PARTE II · FRONT-END REACT

16. Front-end: qué se construye y con qué

Pieza	Elección	Por qué
Biblioteca	React 19	Componentes reutilizables y estado simple con hooks.
Empaquetador	Vite 7	Arranque instantáneo y proxy hacia la API.
Rutas	React Router 7	Pantallas con URL propia y rutas protegidas.
Mapas	Leaflet + OpenStreetMap / Esri	Gratis y sin clave: calles, satélite y relieve.
Íconos	Phosphor Icons	Íconos livianos y consistentes.

```
src/
├── main.jsx, App.jsx   arranque y rutas
├── pages/              una pantalla por archivo (admin/ = panel del administrador)
├── components/         Mapa, Galeria, ChatFlotante, SubirFotos, Layout…
├── ui/                 Modal, Desplegable, avisos
├── context/            auth (sesión), tema (claro/oscuro), i18n (idioma)
├── lib/                api (única puerta a la API), format, geo, hooks…
├── idiomas/            es.js y en.js
└── styles/             base.css y tema.css
```

17. Conectar React con la API

`lib/api.js` es la **única puerta** a la API: agrega el token, convierte los errores en `{ code, message }` y cierra la sesión si la API responde 401. Vite pasa `/api` a la API del puerto 3000 (proxy), por eso la web funciona igual en la PC y en el celular.

```
export async function api(ruta, { method = 'GET', body } = {}) {
  const token = localStorage.getItem('token');
  const res = await fetch(`/api/v1${ruta}`, {
    method,
    headers: { 'Content-Type': 'application/json', ...(token && { Authorization: `Bearer
${token}` }) },
    body: body ? JSON.stringify(body) : undefined,
  });
  …
}
```

18. Pantallas y funciones del turista

18.1 Explorar

Portada con carrusel de fotos, buscador, filtros por categoría y zona, y la distancia de cada lugar.

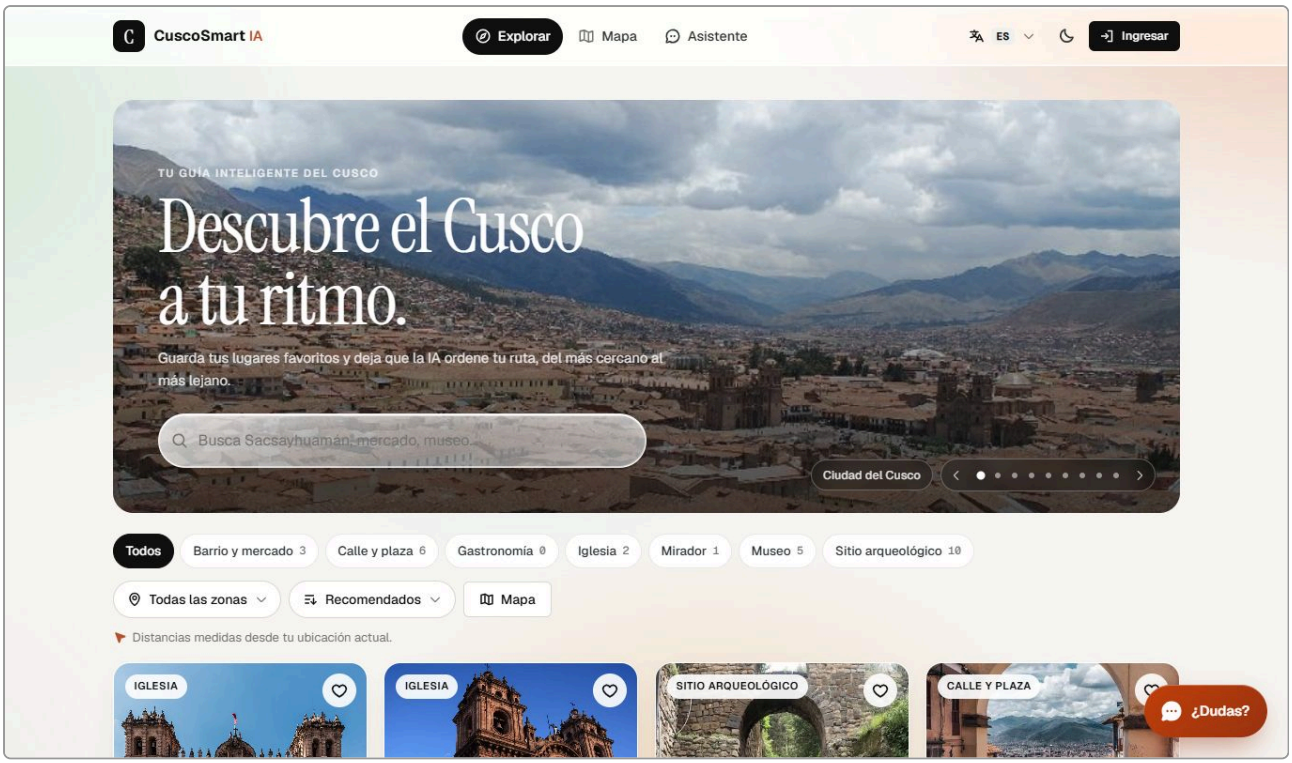


Figura 26. Explorar: carrusel y buscador.

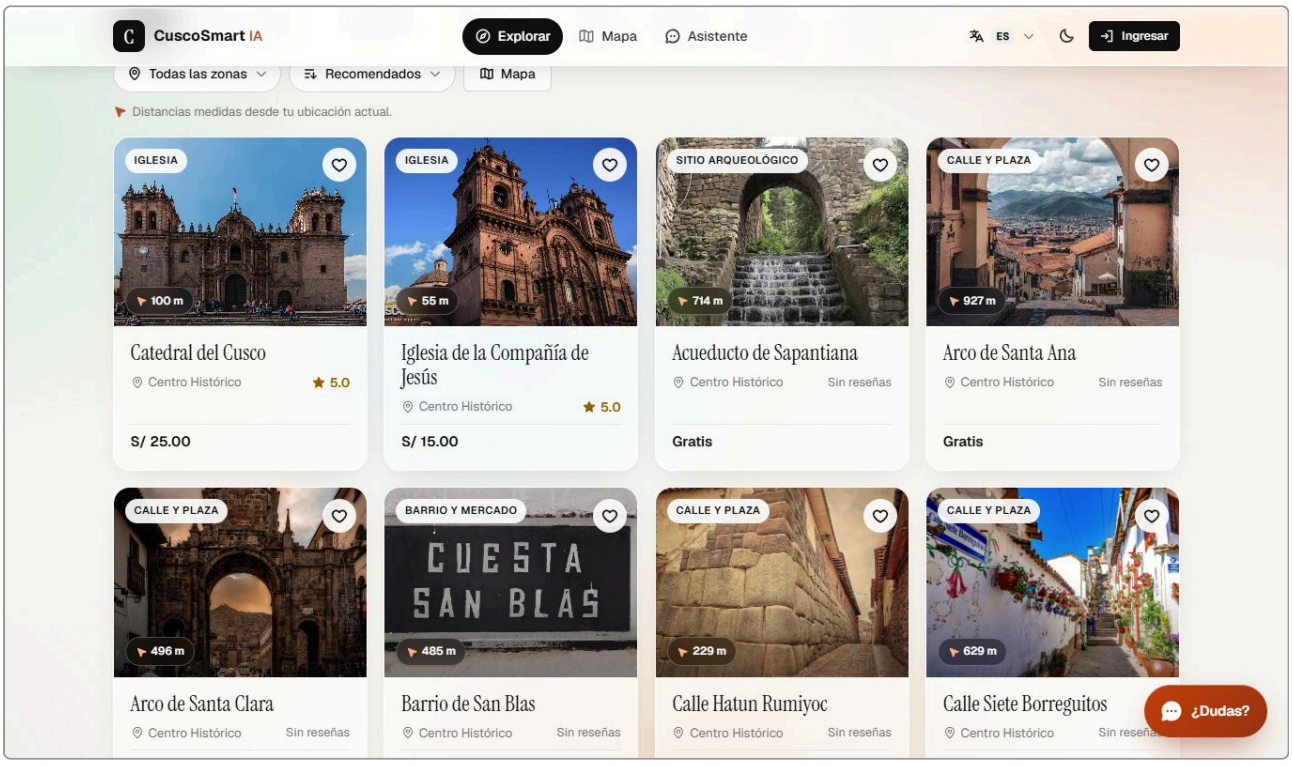


Figura 27. Tarjetas con categoría, calificación, precio y distancia.

18.2 Orden por cercanía y búsqueda con sugerencias

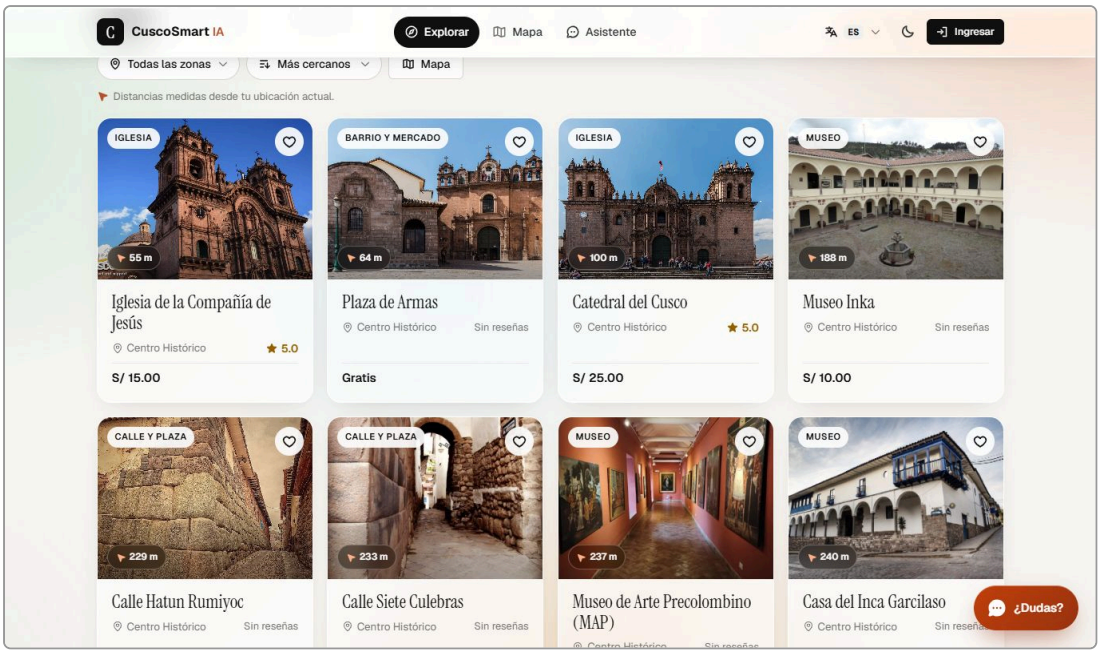


Figura 28. Orden "Más cercanos".

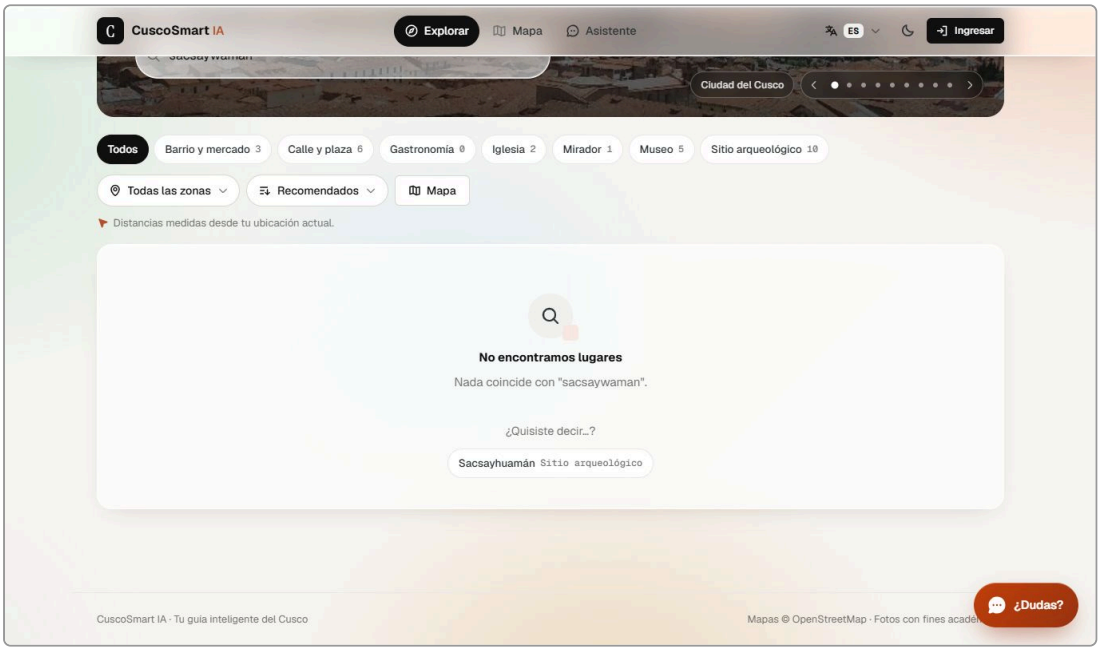


Figura 29. Búsqueda con error de tipeo: la web sugiere "¿Quisiste decir Sacsayhuamán?".

18.3 Detalle del lugar

Galería de hasta 5 fotos, descripción, precio, horarios, mapa, enlaces a Google Maps y 3D, y reseñas.

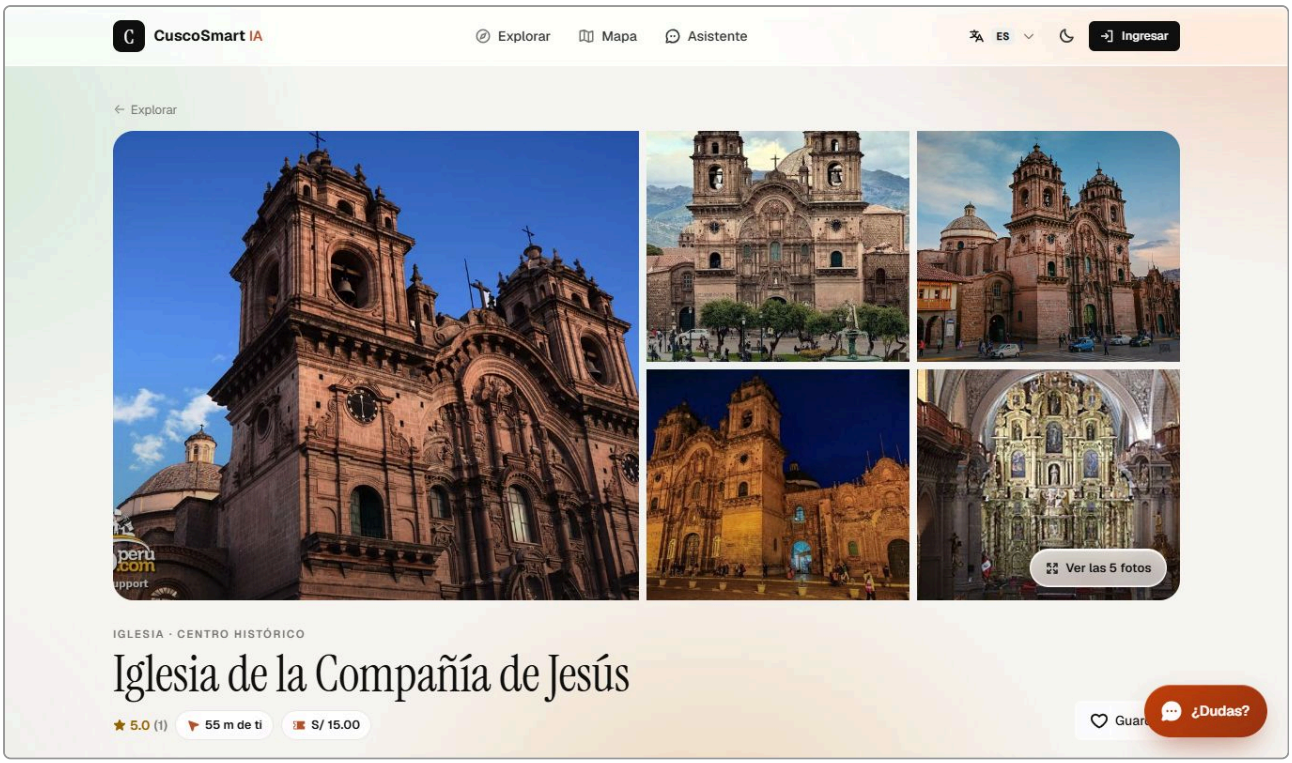


Figura 30. Galería del atractivo.

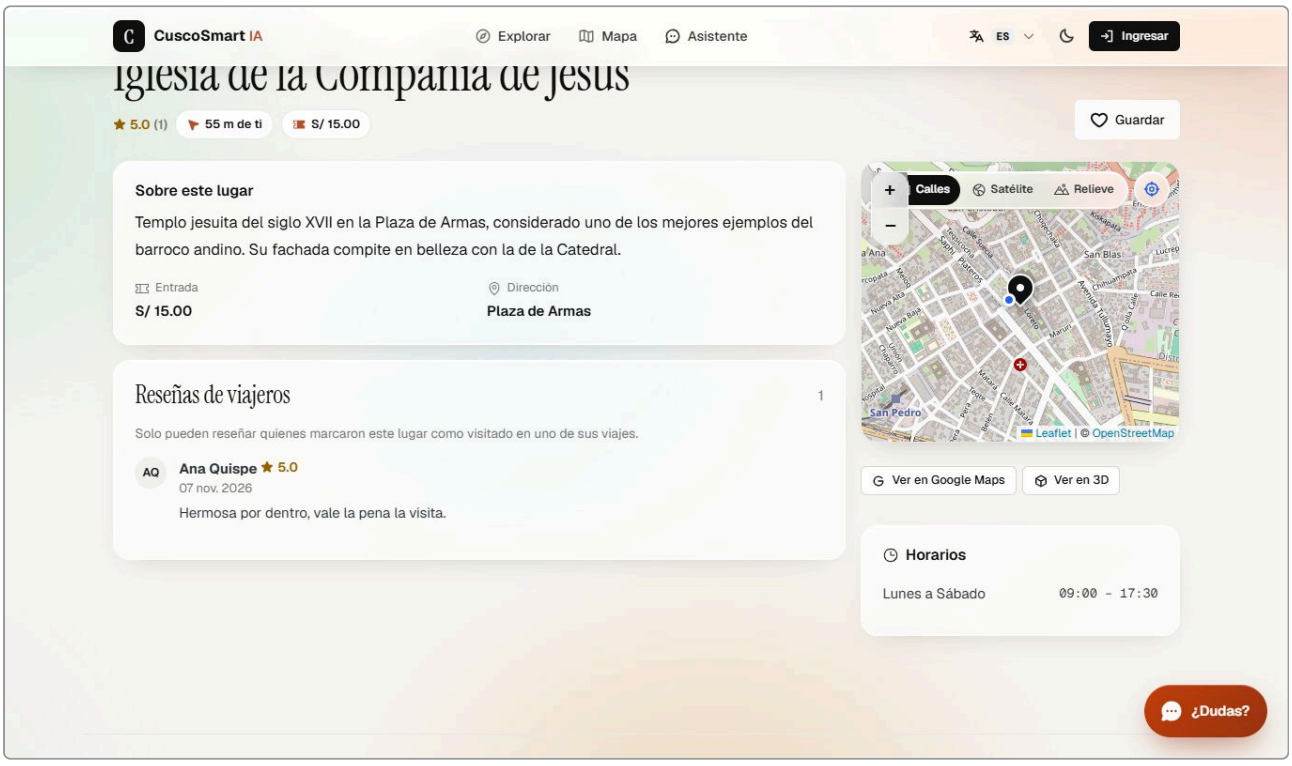


Figura 31. Descripción, horarios, mapa y reseñas.



Figura 32. Visor de fotos con zoom.

18.4 Mapa "cerca de mí"

Lista de lugares del más cercano al más lejano; al elegir uno, el mapa vuela hasta él. Estilos: calles, satélite y relieve.

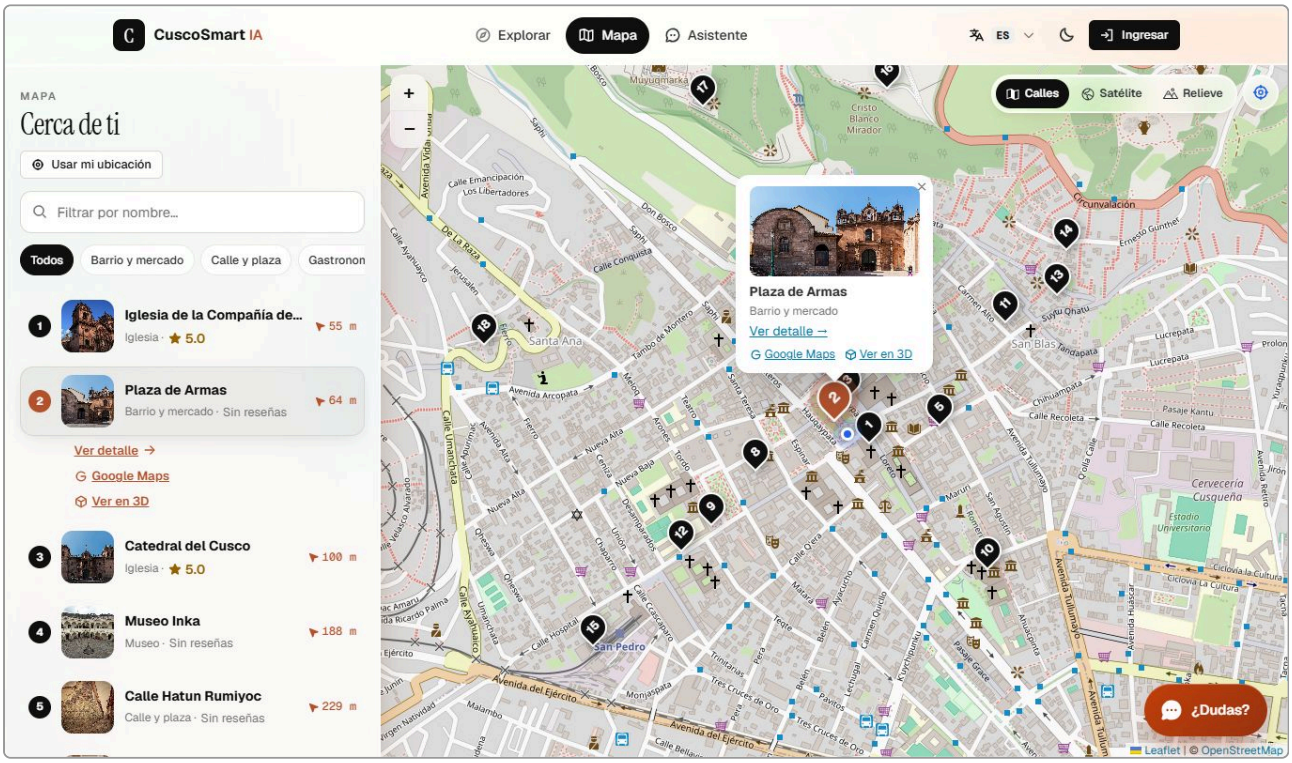


Figura 33. Mapa con la lista por cercanía.

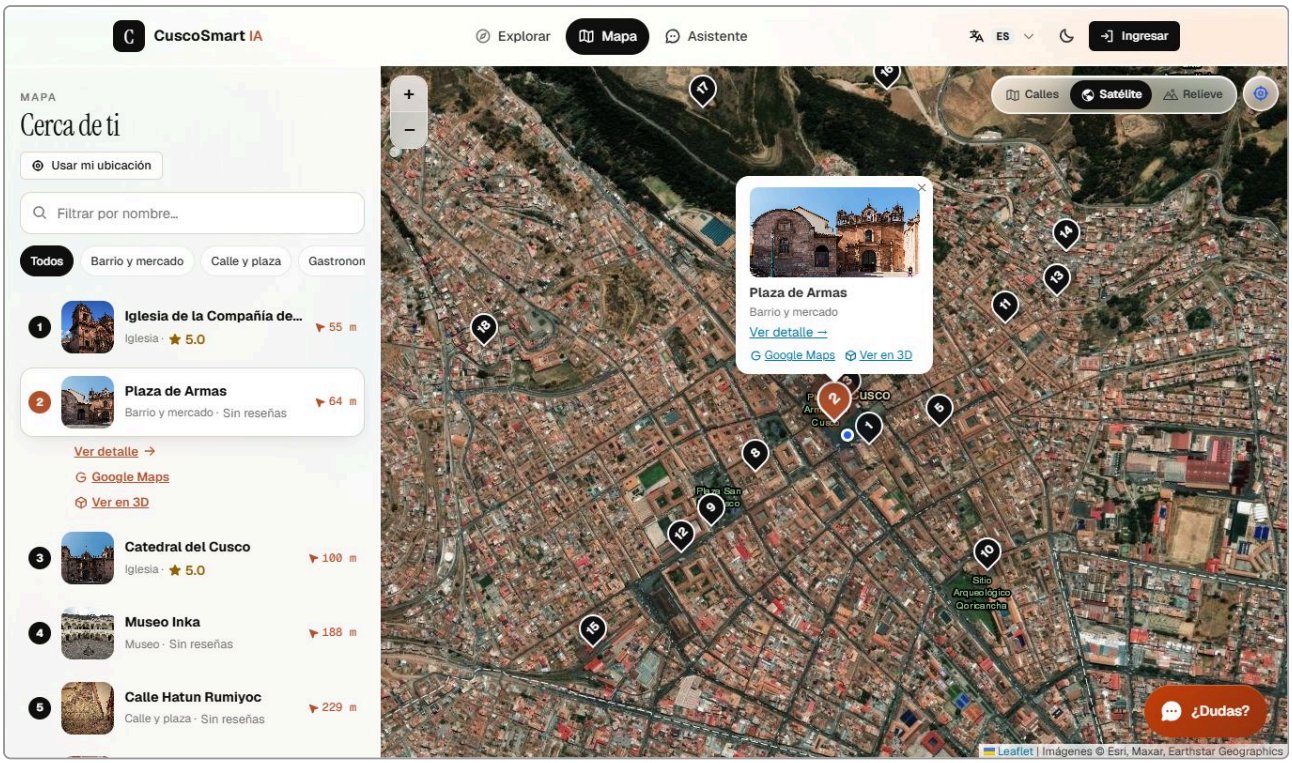


Figura 34. Vista satélite.

18.5 Asistente

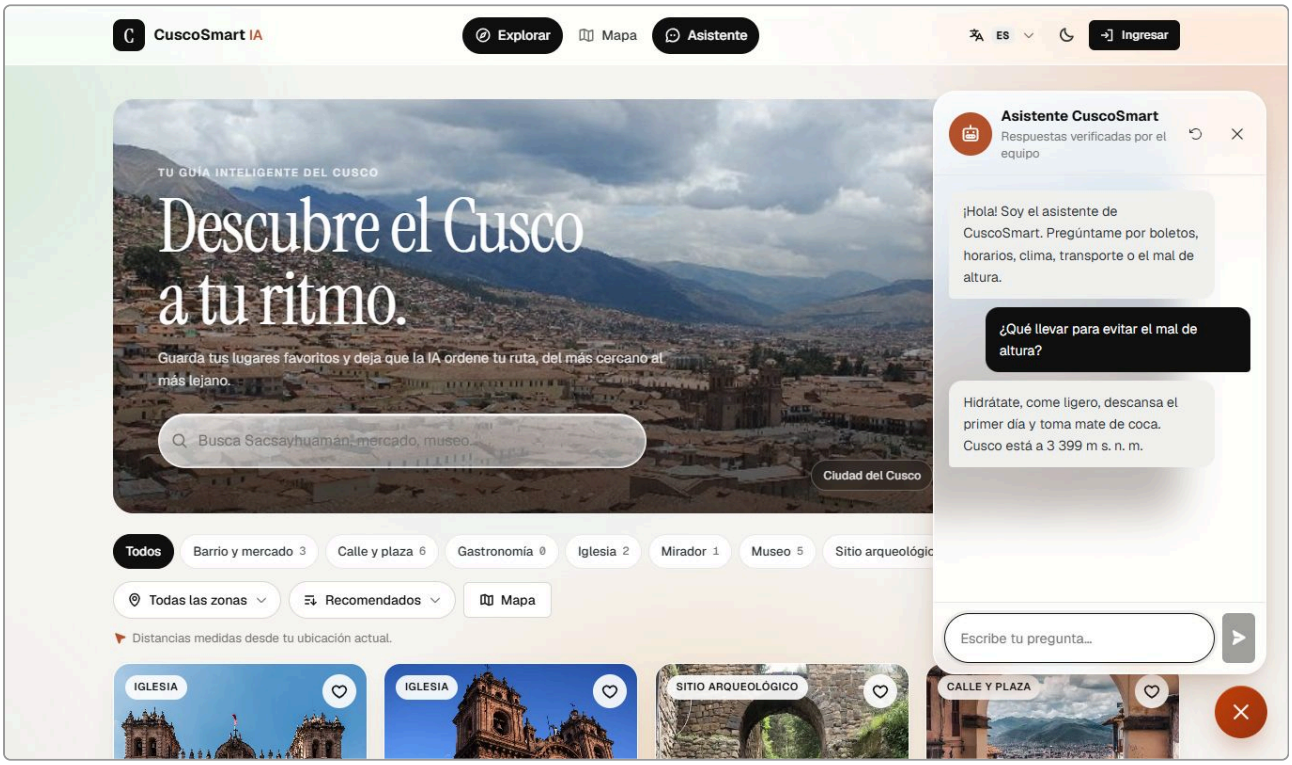


Figura 35. Chat flotante "¿Dudas?" respondiendo una pregunta.

18.6 Ingreso y registro

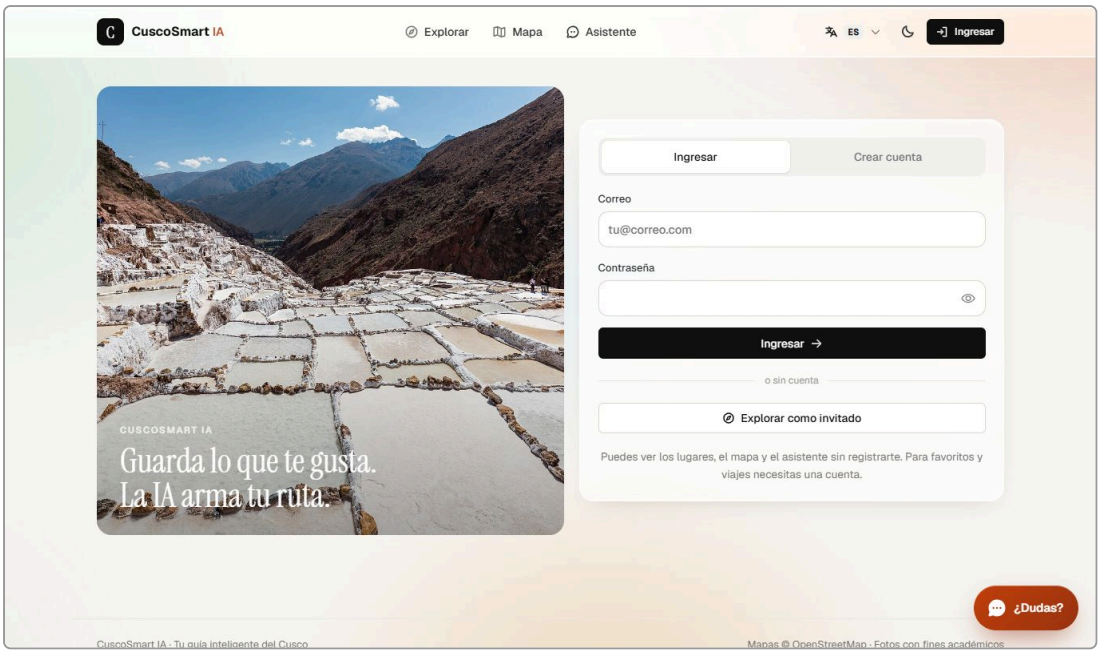


Figura 36. Ingresar (también se puede explorar como invitado).

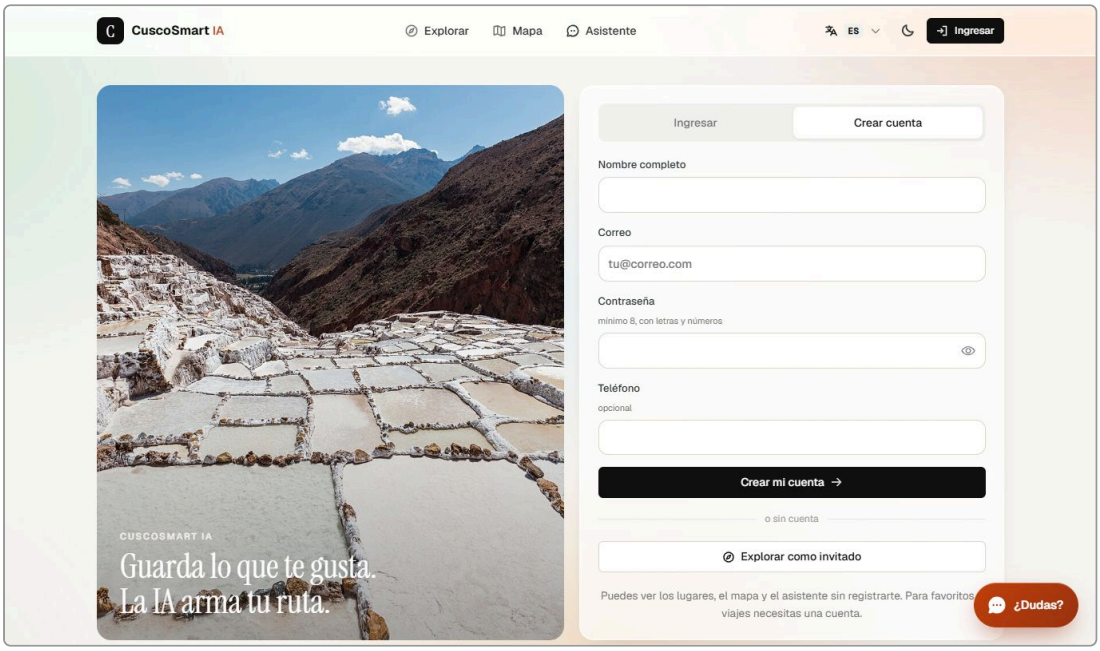


Figura 37. Crear cuenta.

18.7 Favoritos

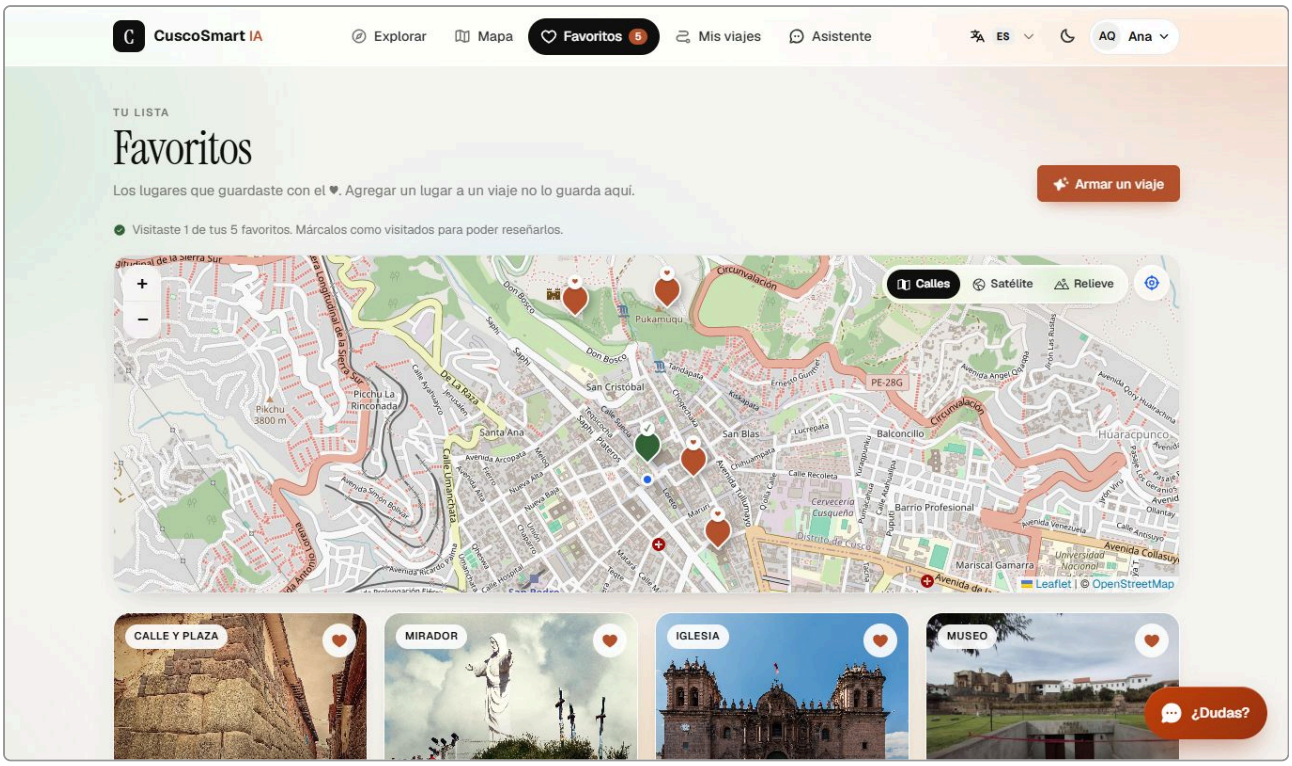


Figura 38. Favoritos en el mapa y en tarjetas.

18.8 Mis viajes y orden con IA

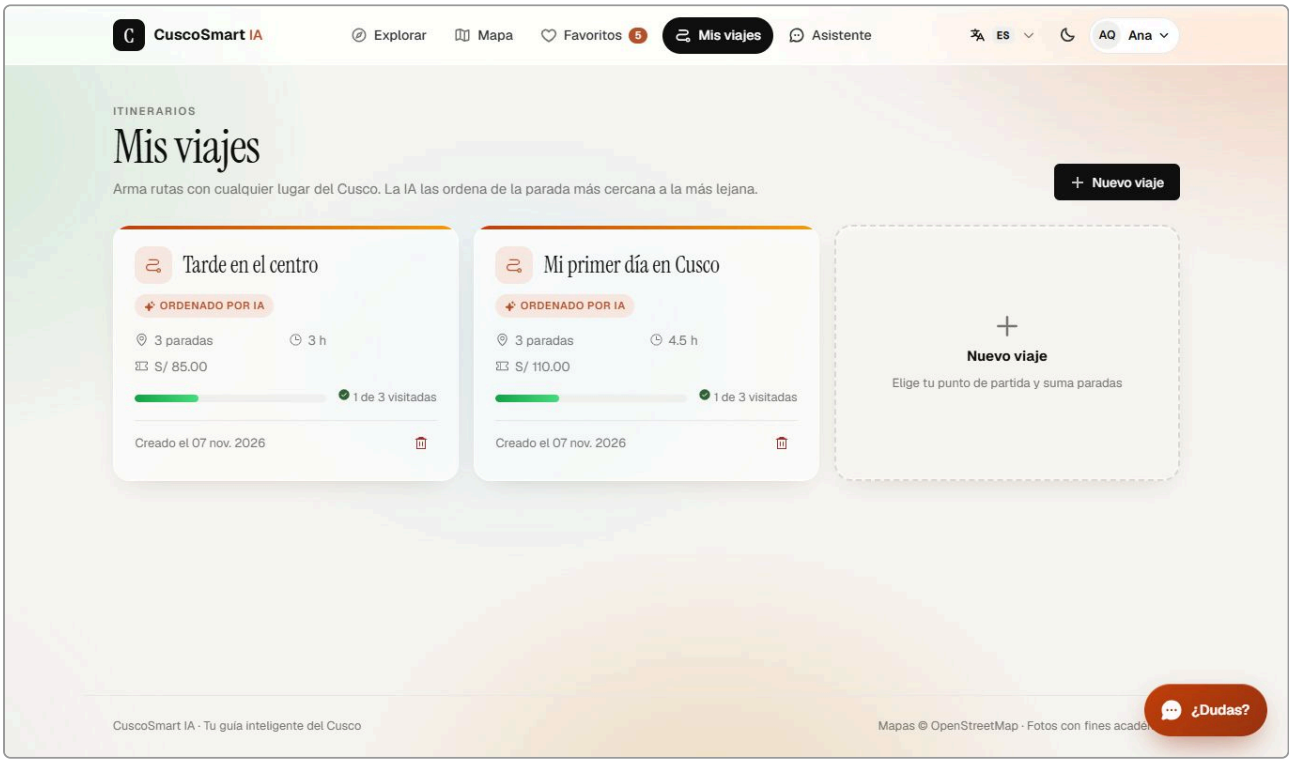


Figura 39. Lista de viajes con avance.

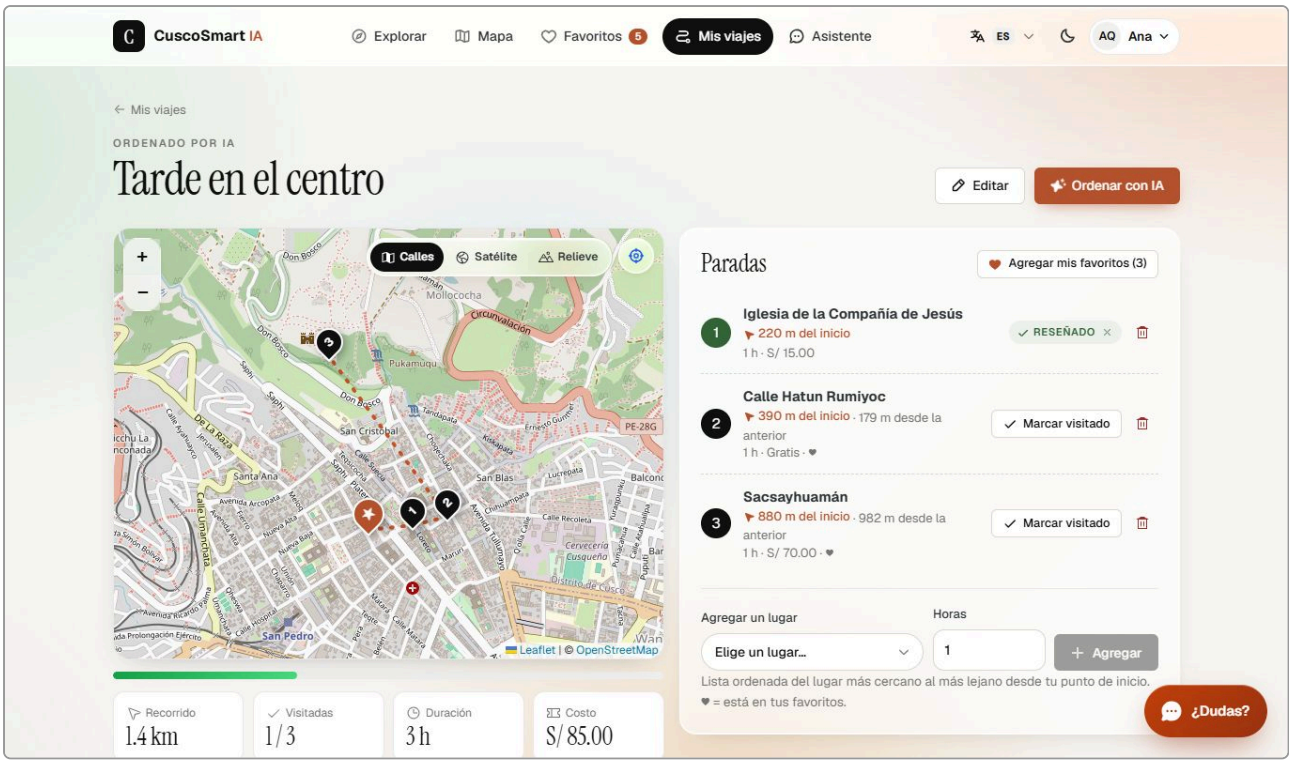


Figura 40. Viaje ordenado por IA: ruta en el mapa, paradas y totales.

18.9 Perfil y "Mi recorrido"

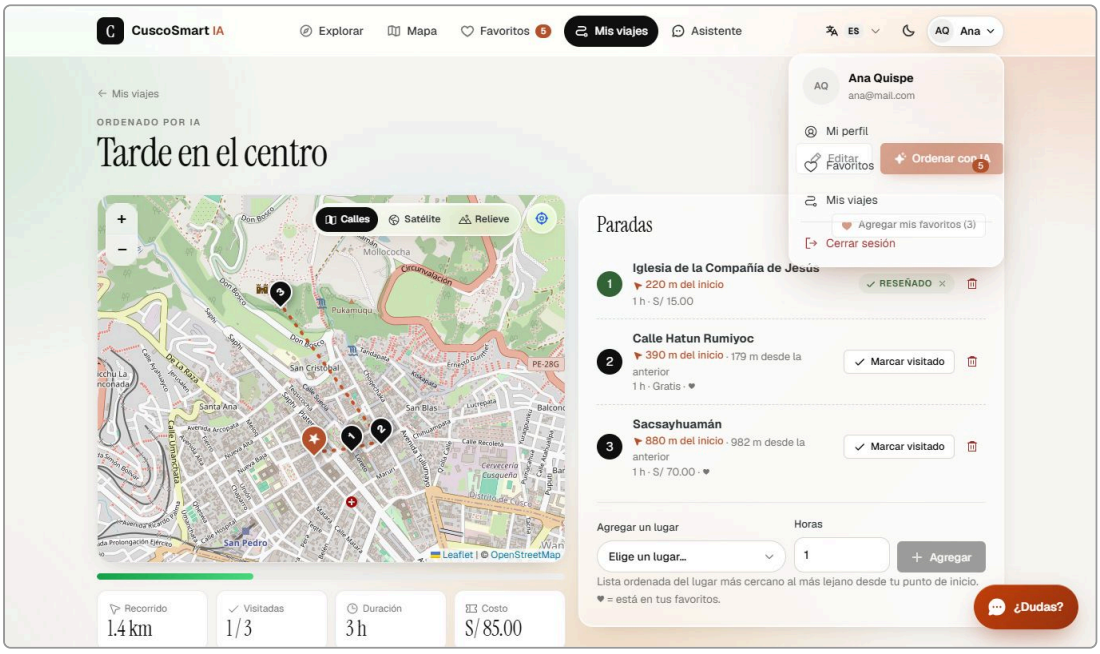


Figura 41. Menú del usuario.

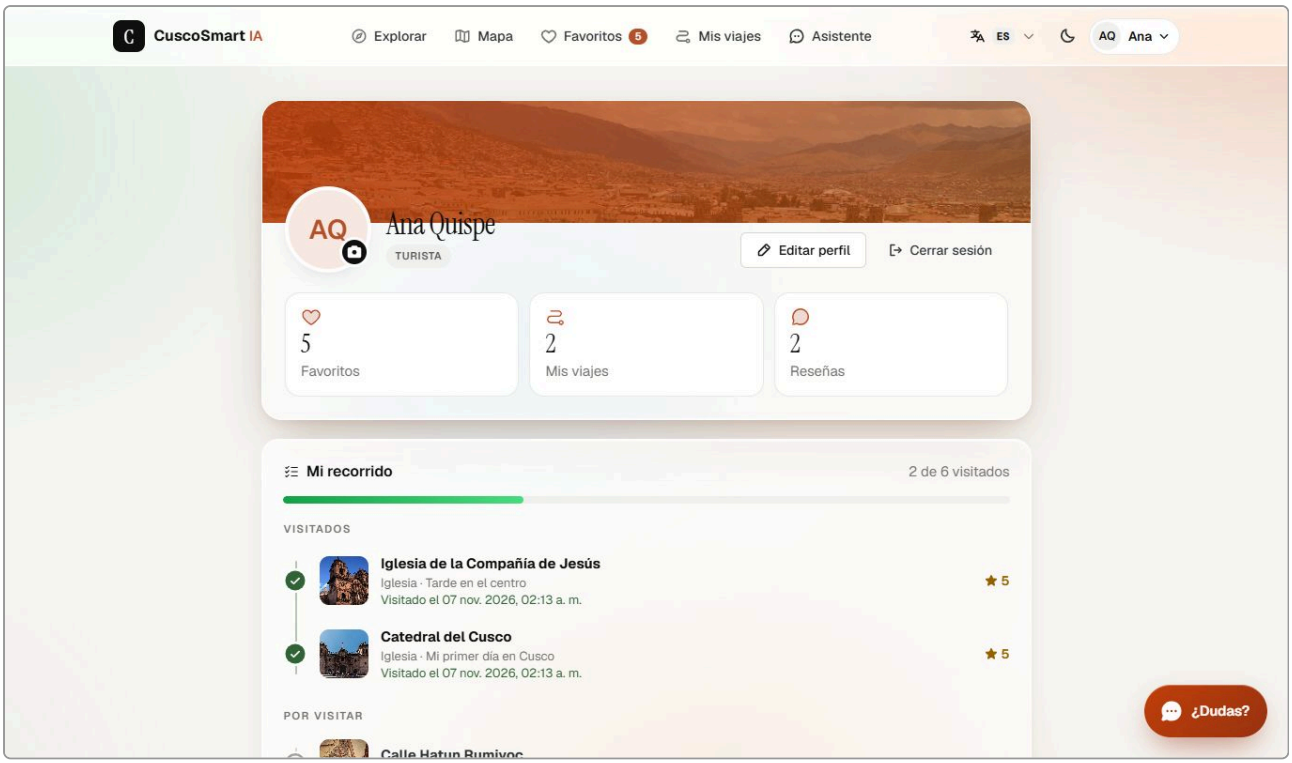


Figura 42. Mi perfil: foto opcional y totales.

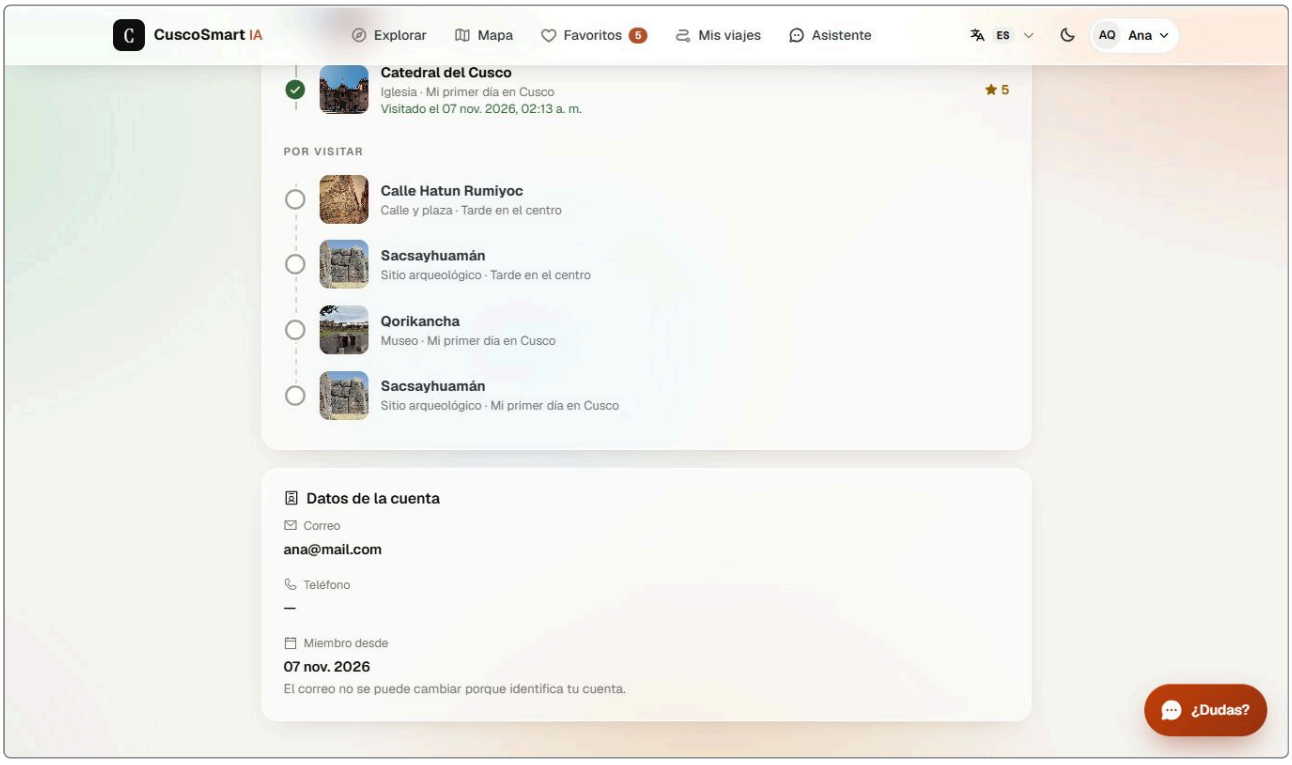


Figura 43. Mi recorrido: lugares visitados y por visitar.

19. Panel del administrador

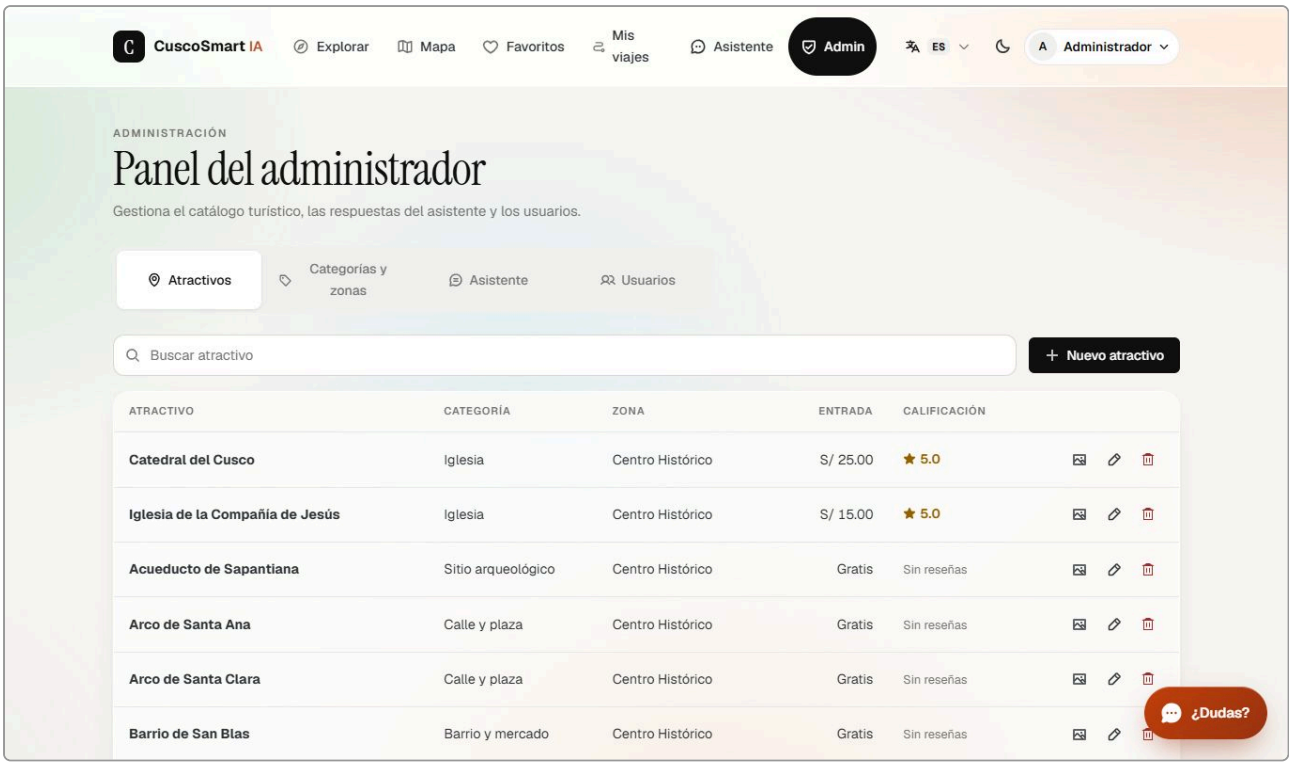


Figura 44. Atractivos: buscar, crear, editar, fotos y borrar.

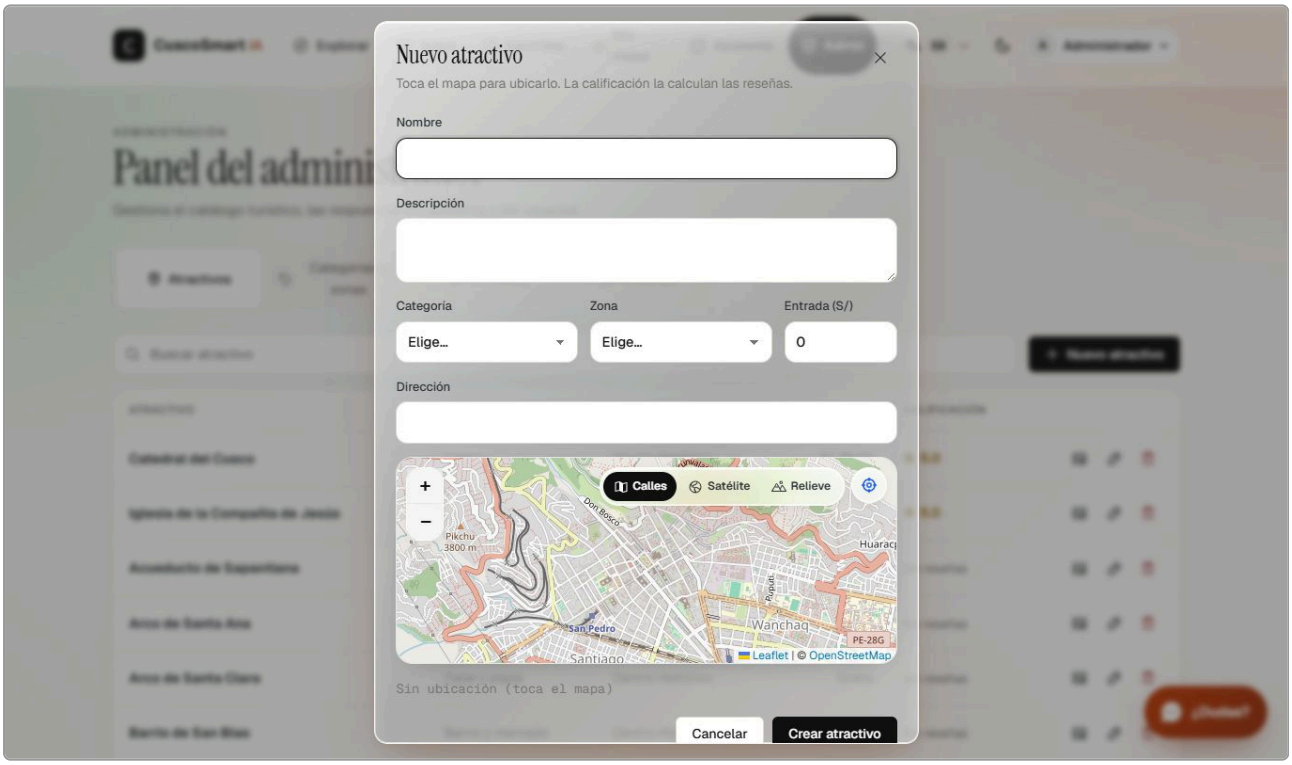


Figura 45. Nuevo atractivo: la ubicación se marca en el mapa.

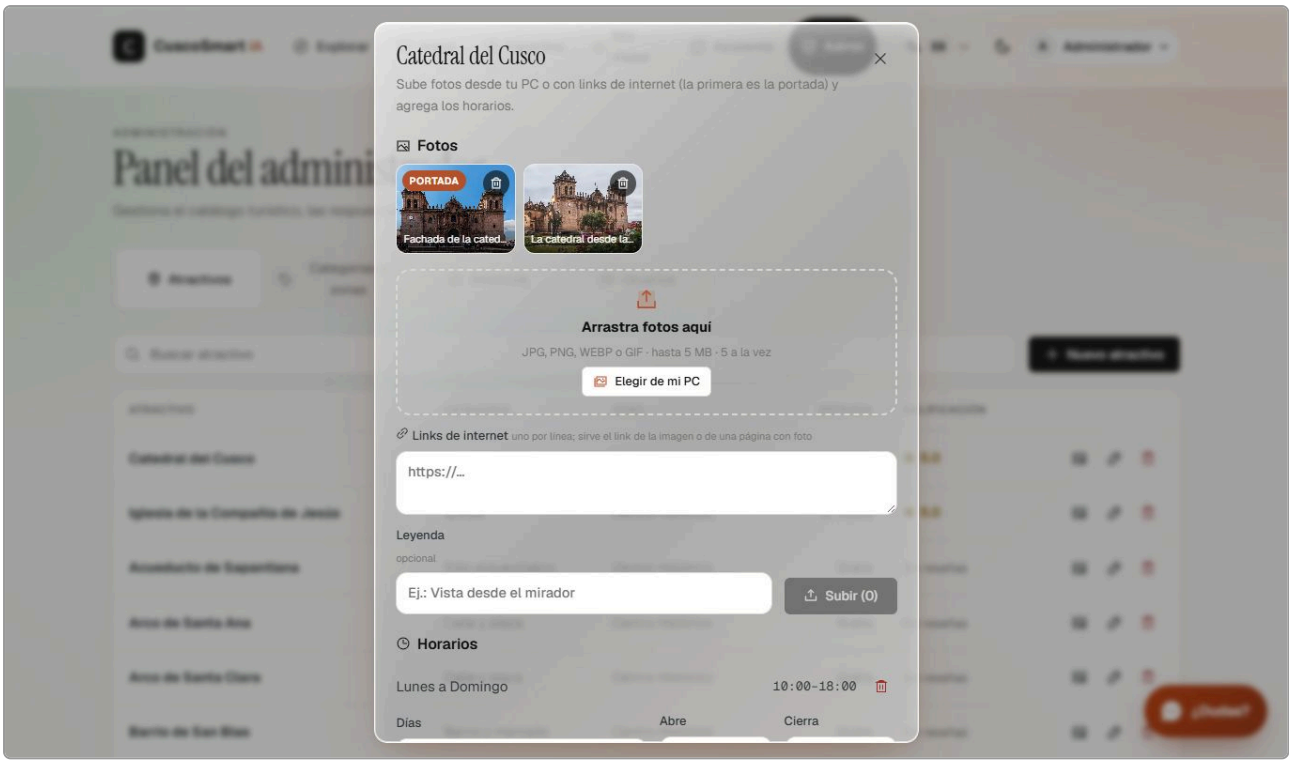


Figura 46. Fotos: portada, subir desde la PC o con links, y horarios.

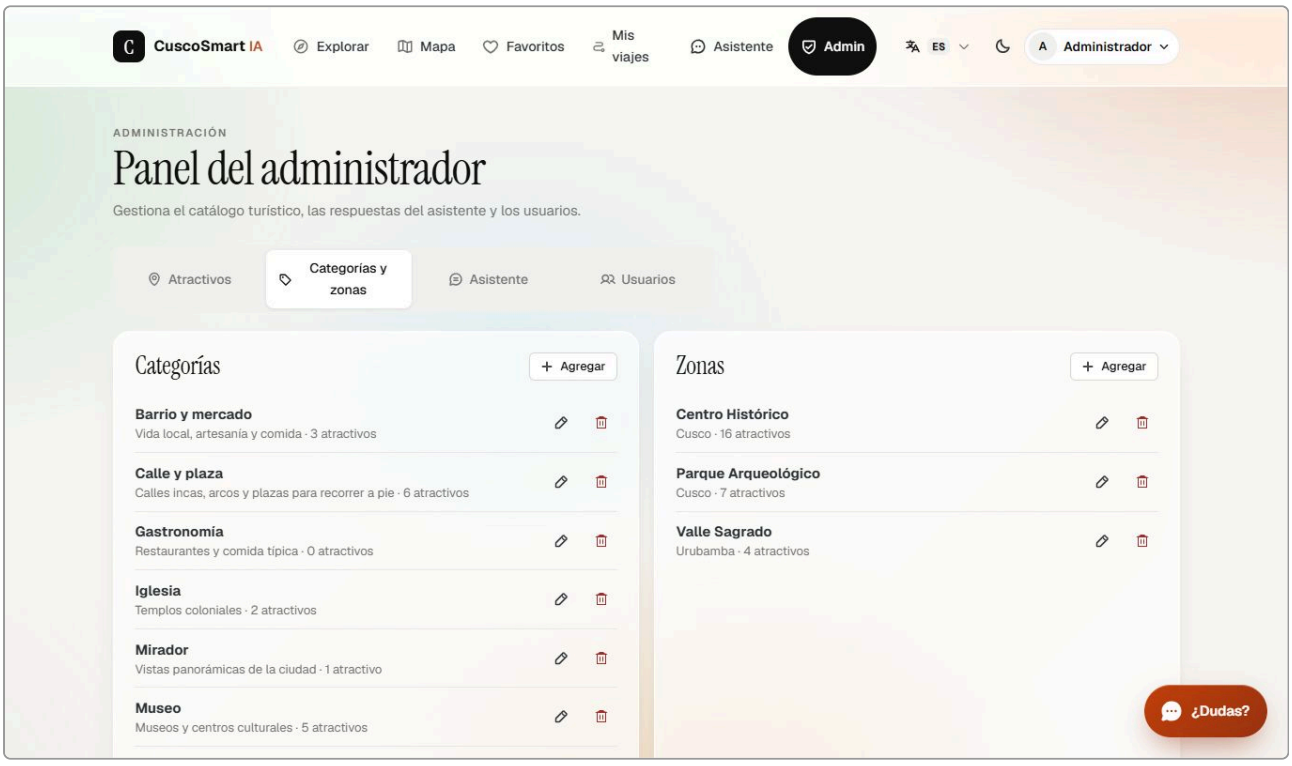


Figura 47. Categorías y zonas.

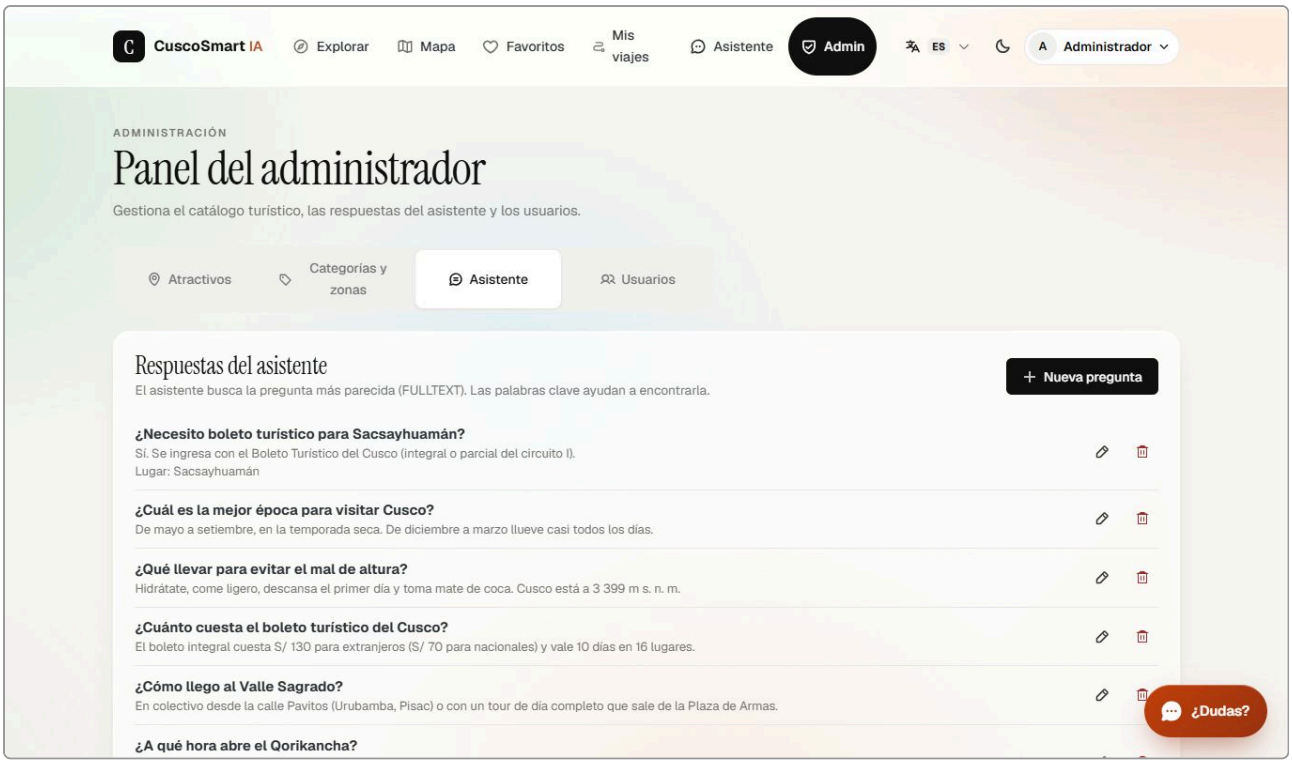


Figura 48. Preguntas frecuentes del asistente.

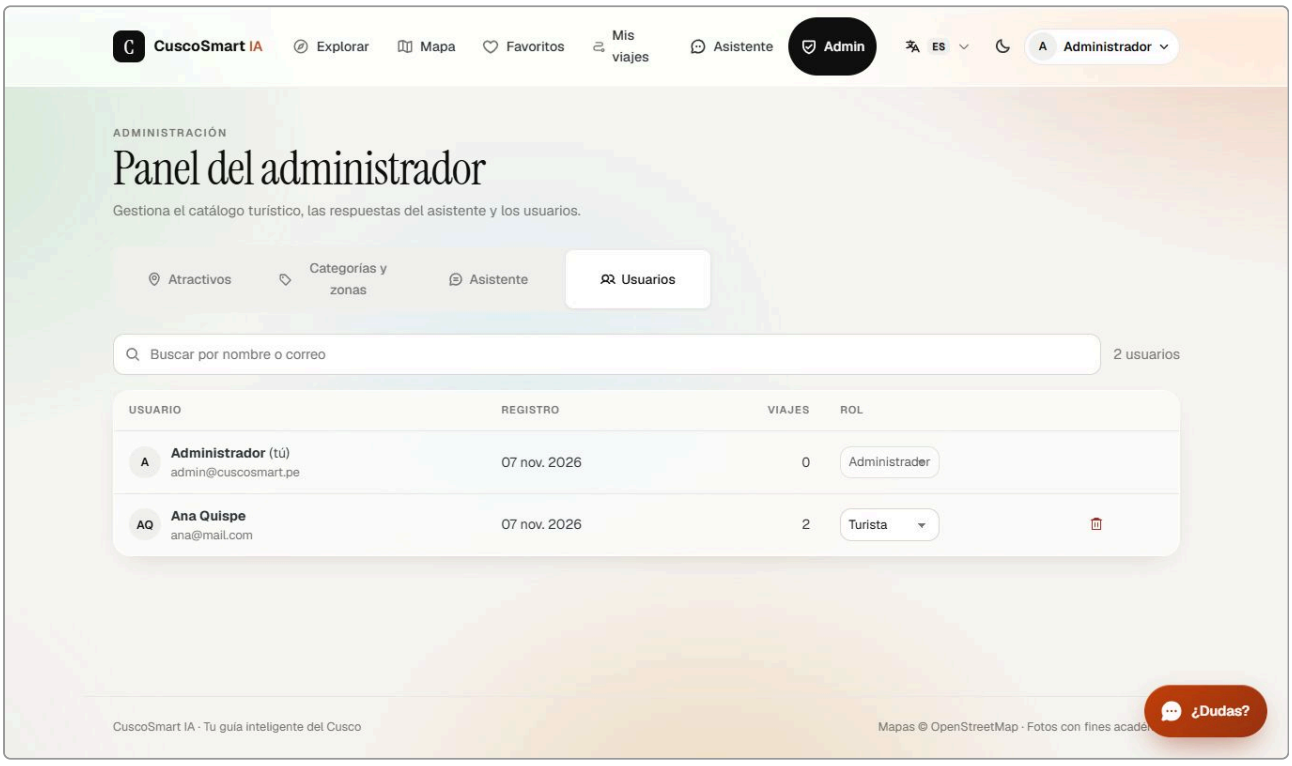


Figura 49. Usuarios: cambiar rol y eliminar.

Las reseñas se moderan desde el detalle de cada lugar: el ADMIN ve un botón para borrar cualquier reseña.

20. Modo oscuro, idioma y celular



Figura 50. Modo oscuro con efecto vidrio.

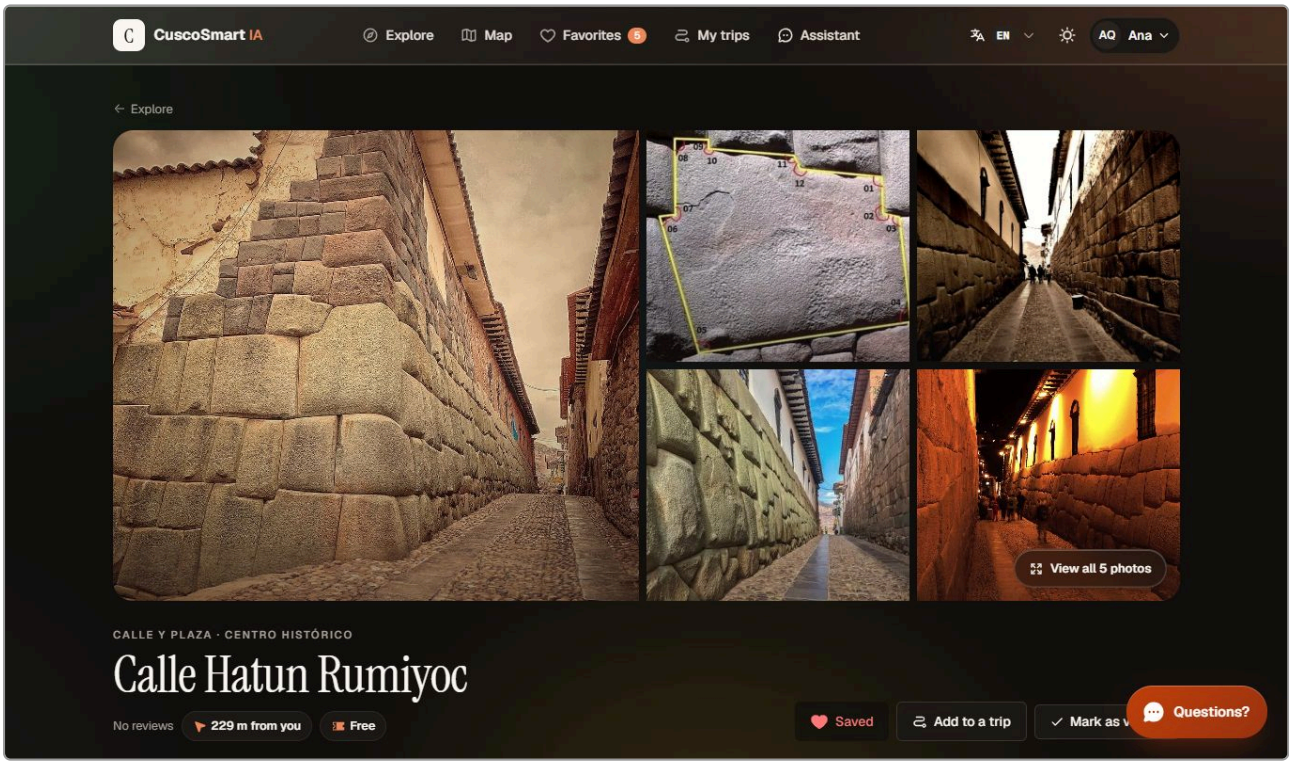


Figura 51. Interfaz en inglés.

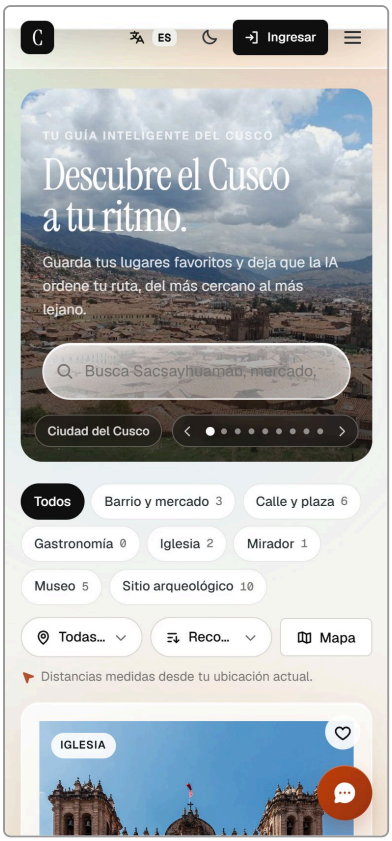


Figura 52. Celular: Explorar.

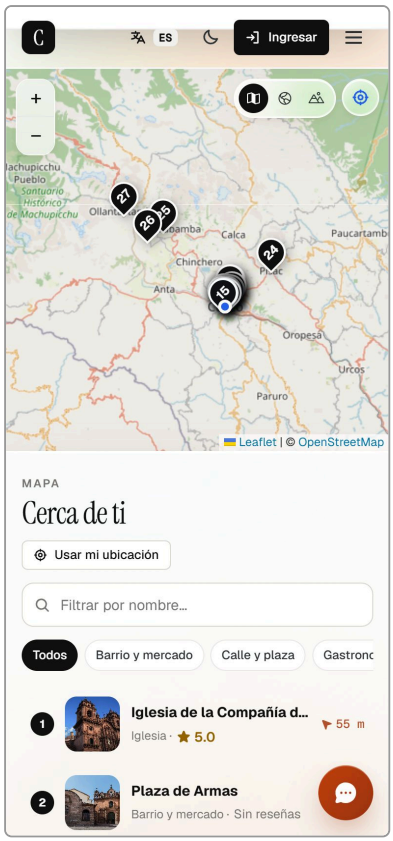


Figura 53. Celular: Mapa.

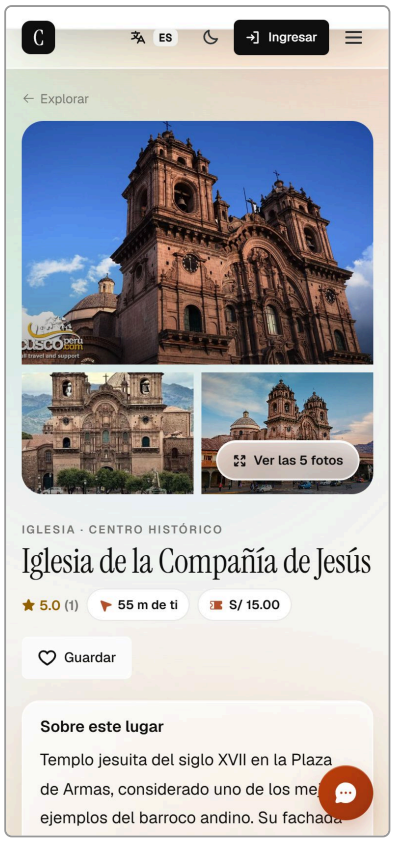


Figura 54. Celular: Detalle.

Parte III · Cierre

21. Cómo ejecutarlo paso a paso

1. XAMPP → **Start** en MySQL.
2. Ventana 1 (API):

```
cd $HOME\Desktop\cuscosmart-api
npm install
Copy-Item .env.example .env      # y pega una clave larga en JWT_SECRET
npm run db:reset
npm run dev
```

3. Ventana 2 (web):

```
cd $HOME\Desktop\cuscosmart-web
npm install
npm run dev
```

4. Abrir `http://localhost:5173`.

Rol	Correo	Contraseña
Administrador	admin@cuscosmart.pe	Admin12345
Turista	ana@mail.com	Turista123

Guía completa con solución de problemas: `docs/INSTALACION.md`.

22. Resumen de pruebas y checklist

Prueba	Resultado
Colección Postman (`npm run test:api`)	**149 peticiones · 456 pruebas aprobadas · 0 fallidas**
Documentación (`npm run docs:verificar`)	**61 de 61 rutas documentadas**
Base vs. diagrama ER	**11 tablas, 12 llaves, cascadas, UNIQUE, CHECK y triggers correctos**
Recorrido E2E en el navegador (Playwright)	**Escritorio y celular sin errores**

☐ La API arranca y `/health` responde con la base conectada.

☐ Login, roles y lo ajeno responden 401 / 403 / 404 según corresponda.

☐ Las reglas de negocio están en procedimientos y triggers, y tienen su prueba.

☐ Swagger documenta todas las rutas.

☐ La web consume la API solo a través de `lib/api.js`.

☐ Funciona en escritorio, celular, modo oscuro y en dos idiomas.

☐ El `.env` no está en GitHub.

23. Historial de Git por fases

Fase	Commit	Qué se logró
0	`chore`	Estructura de carpetas, repositorio y guía.
1	`feat`	Base de datos con procedimientos, triggers y datos de prueba.
2	`feat`	Servidor Express, conexión a MySQL y `/health`.
3	`feat`	Manejo de errores y validación con zod.
4	`feat`	Autenticación JWT y roles.
5	`feat`	Catálogo: categorías, zonas y atractivos.
6	`feat`	Favoritos e itinerarios con orden IA y reseñas verificadas.
7	`feat`	Asistente, preguntas frecuentes, reseñas públicas y usuarios.
8	`test` · `docs`	Colección Postman con pruebas y Swagger.
9	`feat`	Front React con mapa.
10	`feat`	Login con Google, galería, mapa "cerca de mí", chat flotante, modo oscuro e idiomas.
11	`feat`	Paradas sin favoritos, "Mi recorrido", búsqueda tolerante y reseñas propias.
12	`feat`	Marcar visitados sin viaje y armar viaje con favoritos.
13	`feat`	Subir fotos (PC o link), foto de perfil, carrusel, mapa satélite y uso en celular.
—	`refactor`	Un archivo por recurso en cada capa (mismo orden que el proyecto de referencia).

Cada fase se trabajó en una rama `feature/fase-N`, se probó y se unió a `main` con un merge. La guía de cada fase está en `docs/guia/`.

24. Errores frecuentes

Síntoma	Causa	Solución
`ECONNREFUSED` al arrancar	MySQL apagado	XAMPP → Start en MySQL
`Faltan variables en el archivo .env`	No se creó el `.env`	`Copy-Item .env.example .env` y completar `JWT_SECRET`
`Cannot find package …`	Faltan dependencias	`npm install`

La web dice "No se pudo conectar con la API"	API apagada	`npm run dev` en la API
Solo aparecen 14 lugares	Base antigua	`npm run db:reset`
429 "Demasiados intentos"	10 logins fallidos con ese correo	Esperar 15 min o reiniciar la API
`EADDRINUSE :3000`	La API ya está abierta	Usar esa ventana o cerrarla